LibJuno Software Design Document

Table of Contents

1. Introduction . . . 8
   1.1. Purpose . . . 8
   1.2. Scope . . . 8
   1.3. Definitions, Acronyms, and Abbreviations . . . 9
   1.4. References . . . 9
2. System Architecture . . . 9
   2.1. Architectural Overview . . . 9
   2.2. Design Rationale: Why Vtable-Based DI . . . 10
   2.3. Module System Pattern: Root / Derivation / Union . . . 11
   2.4. Design Rationale: Root/Derivation/Union Layering . . . 11
   2.5. Subsystem Decomposition . . . 11
   2.6. Module Dependency Overview . . . 12
3. Detailed Design . . . 13
4. Status Module . . . 13
   4.1. Purpose . . . 13
   4.2. Data Structures . . . 13
   4.3. Interface Design . . . 14
   4.4. Algorithm Descriptions . . . 14
   4.5. Error Handling . . . 14
   4.6. Design Rationale . . . 14
   4.7. Requirements Traceability . . . 14
5. Module System . . . 15
   5.1. Purpose . . . 15
   5.2. Data Structures . . . 15
      5.2.1. Module Root (`JUNO_MODULE_ROOT`) . . . 15
      5.2.2. Module Derivation (`JUNO_MODULE_DERIVE`) . . . 15
      5.2.3. Module Union (`JUNO_MODULE`) . . . 15
      5.2.4. Trait Root (`JUNO_TRAIT_ROOT`) . . . 16
      5.2.5. Result Types (`JUNO_MODULE_RESULT`) . . . 16
      5.2.6. Option Types (`JUNO_MODULE_OPTION`) . . . 16
   5.3. Interface Design . . . 16
      5.3.1. Construction Macros . . . 16
      5.3.2. Access Macros . . . 16
      5.3.3. Result/Option Macros . . . 17
   5.4. Algorithm Descriptions . . . 17
   5.5. Error Handling . . . 17

5.6. Design Rationale . . . 17
5.7. Requirements Traceability . . . 18
6. Macros Module . . . 18
6.1. Purpose . . . 18
6.2. Data Structures . . . 18
6.3. Interface Design . . . 18
6.4. Algorithm Descriptions . . . 18
6.5. Error Handling . . . 19
6.6. Design Rationale . . . 19
6.7. Requirements Traceability . . . 19
7. Common Types Module . . . 19
7.1. Purpose . . . 19
7.2. Data Structures . . . 19
7.3. Interface Design . . . 20
7.4. Algorithm Descriptions . . . 20
7.5. Error Handling . . . 20
7.6. Design Rationale . . . 20
7.7. Requirements Traceability . . . 20
8. Pointer Module . . . 20
8.1. Purpose . . . 21
8.2. Data Structures . . . 21
8.2.1. `JUNO_POINTER_T` (Trait Root) . . . 21
8.2.2. `JUNO_POINTER_API_T` . . . 21
8.2.3. `JUNO_VALUE_POINTER_API_T` . . . 21
8.2.4. Result/Option Types . . . 21
8.3. Interface Design . . . 22
8.4. Algorithm Descriptions . . . 22
8.5. Error Handling . . . 22
8.6. Design Rationale . . . 22
8.7. Requirements Traceability . . . 23
9. Memory Module . . . 23
9.1. Purpose . . . 23
9.2. Data Structures . . . 23
9.2.1. Abstract Allocator Root (`JUNO_MEMORY_ALLOC_ROOT_T`) . . . 23
9.2.2. Allocator API (`JUNO_MEMORY_ALLOC_API_T`) . . . 23
9.2.3. Block Allocator Derivation (`JUNO_MEMORY_ALLOC_BLOCK_T`) . . . 24
9.2.4. Block Metadata (`JUNO_MEMORY_BLOCK_METADATA_T`) . . . 24
9.3. Interface Design . . . 24
9.3.1. Storage Declaration Macros . . . 24
9.4. Algorithm Descriptions . . . 24
9.4.1. Block Get (Allocation) . . . 25

9.4.2. Block Put (Deallocation) . . . 25
9.4.3. Block Update . . . 25
9.5. Error Handling . . . 25
9.6. Design Rationale . . . 25
9.7. Requirements Traceability . . . 26
10. Array Module . . . 26
10.1. Purpose . . . 26
10.2. Data Structures . . . 26
10.2.1. Array Root (`JUNO_DS_ARRAY_ROOT_T`) . . . 26
10.2.2. Array API (`JUNO_DS_ARRAY_API_T`) . . . 26
10.3. Interface Design . . . 26
10.4. Algorithm Descriptions . . . 27
10.5. Error Handling . . . 27
10.6. Design Rationale . . . 27
10.7. Requirements Traceability . . . 27
11. Heap Module . . . 28
11.1. Purpose . . . 28
11.2. Data Structures . . . 28
11.2.1. Heap Root (`JUNO_DS_HEAP_ROOT_T`) . . . 28
11.2.2. Heap Pointer API (`JUNO_DS_HEAP_POINTER_API_T`) . . . 28
11.2.3. Heap API (`JUNO_DS_HEAP_API_T`) . . . 28
11.2.4. Helper Result/Option Types . . . 28
11.3. Interface Design . . . 29
11.4. Algorithm Descriptions . . . 29
11.4.1. Insert . . . 29
11.4.2. Pop . . . 29
11.4.3. Heapify . . . 29
11.4.4. SiftDown . . . 30
11.5. Error Handling . . . 30
11.6. Design Rationale . . . 30
11.7. Requirements Traceability . . . 30
12. Queue Module . . . 30
12.1. Purpose . . . 30
12.2. Data Structures . . . 31
12.2.1. Queue Root (`JUNO_DS_QUEUE_ROOT_T`) . . . 31
12.2.2. Queue API (`JUNO_DS_QUEUE_API_T`) . . . 31
12.3. Interface Design . . . 31
12.4. Algorithm Descriptions . . . 32
12.4.1. Enqueue . . . 32
12.4.2. Dequeue . . . 32
12.4.3. Peek . . . 32

12.5. Error Handling . . . 32
12.6. Design Rationale . . . 32
12.7. Requirements Traceability . . . 32
13. Stack Module . . . 33
13.1. Purpose . . . 33
13.2. Data Structures . . . 33
13.2.1. Stack Root (`JUNO_DS_STACK_ROOT_T`) . . . 33
13.2.2. Stack API (`JUNO_DS_STACK_API_T`) . . . 33
13.3. Interface Design . . . 33
13.4. Algorithm Descriptions . . . 34
13.4.1. Push . . . 34
13.4.2. Pop . . . 34
13.4.3. Peek . . . 34
13.5. Error Handling . . . 34
13.6. Design Rationale . . . 34
13.7. Requirements Traceability . . . 34
14. Map Module . . . 35
14.1. Purpose . . . 35
14.2. Data Structures . . . 35
14.2.1. Map Root (`JUNO_MAP_ROOT_T`) . . . 35
14.2.2. Hashable Pointer API (`JUNO_MAP_HASHABLE_POINTER_API_T`) . . . 35
14.2.3. Map API (`JUNO_MAP_API_T`) . . . 35
14.3. Interface Design . . . 35
14.4. Algorithm Descriptions . . . 36
14.4.1. Set (Insert/Update) . . . 36
14.4.2. Get (Lookup) . . . 36
14.4.3. Remove . . . 36
14.5. Error Handling . . . 36
14.6. Design Rationale . . . 37
14.7. Requirements Traceability . . . 37
15. CRC Module . . . 37
15.1. Purpose . . . 37
15.2. Data Structures . . . 37
15.2.1. Seed Constants . . . 37
15.3. Interface Design . . . 38
15.3.1. ZIP Streaming Note . . . 38
15.4. Algorithm Descriptions . . . 38
15.5. Error Handling . . . 39
15.6. Design Rationale . . . 39
15.7. Requirements Traceability . . . 39
16. Hash Module . . . 39

16.1. Purpose . . . 39
16.2. Data Structures . . . 39
16.3. Interface Design . . . 39
16.4. Algorithm Descriptions . . . 40
16.4.1. djb2 . . . 40
16.5. Error Handling . . . 40
16.6. Design Rationale . . . 40
16.7. Requirements Traceability . . . 40
17. Math Module . . . 40
17.1. Purpose . . . 40
17.2. Data Structures . . . 41
17.2.1. Scalar Types . . . 41
17.2.2. Vector Types . . . 41
17.2.3. Quaternion Types . . . 41
17.2.4. Result Types . . . 41
17.3. Interface Design . . . 41
17.3.1. Generic Math Macros . . . 41
17.3.2. Vector Operations . . . 42
17.4. Algorithm Descriptions . . . 42
17.4.1. Dot Product . . . 42
17.4.2. Cross Product (3D) . . . 42
17.4.3. Norm . . . 42
17.4.4. Normalize . . . 43
17.5. Error Handling . . . 43
17.6. Design Rationale . . . 43
17.7. Requirements Traceability . . . 43
18. Time Module . . . 43
18.1. Purpose . . . 43
18.2. Data Structures . . . 44
18.2.1. Timestamp (`JUNO_TIMESTAMP_T`) . . . 44
18.2.2. Time Root (`JUNO_TIME_ROOT_T`) . . . 44
18.2.3. Time API (`JUNO_TIME_API_T`) . . . 44
18.2.4. Result Types . . . 44
18.3. Interface Design . . . 45
18.3.1. LibJuno-Provided Functions . . . 45
18.4. Algorithm Descriptions . . . 45
18.4.1. AddTime . . . 45
18.4.2. SubtractTime . . . 45
18.4.3. TimestampToNanos . . . 45
18.4.4. DoubleToTimestamp . . . 46
18.5. Error Handling . . . 46

18.6. Design Rationale . . . 46
18.7. Requirements Traceability . . . 46
19. IO Module . . . 47
19.1. Purpose . . . 47
19.2. Data Structures . . . 47
19.2.1. Async IO Root (`JUNO_ASYNC_IO_ROOT_T`) . . . 47
19.2.2. Async IO API (`JUNO_ASYNC_IO_API_T`) . . . 47
19.2.3. I2C IO Root and Messages . . . 48
19.2.4. I2C IO API (`JUNO_I2C_IO_API_T`) . . . 48
19.2.5. I2C Helper Macros . . . 48
19.2.6. SPI IO Root and API . . . 48
19.3. Interface Design . . . 48
19.4. Algorithm Descriptions . . . 49
19.5. Error Handling . . . 49
19.6. Design Rationale . . . 49
19.7. Requirements Traceability . . . 49
20. Log Module . . . 50
20.1. Purpose . . . 50
20.2. Data Structures . . . 50
20.2.1. Log Root (`JUNO_LOG_ROOT_T`) . . . 50
20.2.2. Log API (`JUNO_LOG_API_T`) . . . 50
20.3. Interface Design . . . 50
20.4. Algorithm Descriptions . . . 50
20.5. Error Handling . . . 50
20.6. Design Rationale . . . 51
20.7. Requirements Traceability . . . 51
21. Mutex Module . . . 51
21.1. Purpose . . . 51
21.2. Data Structures . . . 51
21.2.1. Mutex Root (`JUNO_MP_MTX_ROOT_T`) . . . 51
21.2.2. Mutex API (`JUNO_MP_MTX_API_T`) . . . 51
21.3. Interface Design . . . 52
21.4. Algorithm Descriptions . . . 52
21.5. Error Handling . . . 52
21.6. Design Rationale . . . 52
21.7. Requirements Traceability . . . 52
22. State Machine Module . . . 53
22.1. Purpose . . . 53
22.2. Data Structures . . . 53
22.2.1. SM Root (`JUNO_SM_ROOT_T`) . . . 53
22.2.2. State Root (`JUNO_SM_STATE_ROOT_T`) . . . 53

22.2.3. State API (JUNO_SM_STATE_API_T) . . . 53
22.2.4. Result/Option Types . . . 53
22.3. Interface Design . . . 54
22.4. Algorithm Descriptions . . . 54
22.4.1. State Machine Execution Loop (User-Driven) . . . 54
22.4.2. Transition . . . 54
22.5. Error Handling . . . 55
22.6. Design Rationale . . . 55
22.7. Requirements Traceability . . . 55
23. Software Bus (Broker) Module . . . 55
23.1. Purpose . . . 55
23.2. Data Structures . . . 56
23.2.1. Broker Root (JUNO_SB_BROKER_ROOT_T) . . . 56
23.2.2. Pipe (JUNO_SB_PIPE_T) . . . 56
23.2.3. Message ID (JUNO_SB_MID_T) . . . 56
23.2.4. Broker API (JUNO_SB_BROKER_API_T) . . . 56
23.3. Interface Design . . . 56
23.4. Algorithm Descriptions . . . 57
23.4.1. Publish . . . 57
23.4.2. RegisterSubscriber . . . 57
23.5. Error Handling . . . 57
23.6. Design Rationale . . . 57
23.7. Requirements Traceability . . . 58
24. Application Module . . . 58
24.1. Purpose . . . 58
24.2. Data Structures . . . 58
24.2.1. Application Root (JUNO_APP_ROOT_T) . . . 58
24.2.2. Application API (JUNO_APP_API_T) . . . 59
24.3. Interface Design . . . 59
24.4. Algorithm Descriptions . . . 59
24.5. Error Handling . . . 59
24.6. Design Rationale . . . 59
24.7. Requirements Traceability . . . 59
25. Scheduler Module . . . 60
25.1. Purpose . . . 60
25.2. Data Structures . . . 60
25.2.1. Scheduler Root (JUNO_SCH_ROOT_T) . . . 60
25.2.2. Scheduler API (JUNO_SCH_API_T) . . . 60
25.2.3. Schedule Table Macro . . . 60
25.3. Interface Design . . . 61
25.4. Algorithm Descriptions . . . 61

25.4.1. Execute (One Major Frame) . . . 61
25.4.2. GetMajorFramePeriod . . . 61
25.4.3. Flattened 2D Indexing . . . 61
25.5. Error Handling . . . 62
25.6. Design Rationale . . . 62
25.7. Requirements Traceability . . . 62
26. Data Design . . . 62
26.1. Common Types . . . 62
26.2. Memory Ownership Model . . . 63
26.3. Design Rationale: Result/Option Types and Error Handling . . . 63
27. Interface Design . . . 64
27.1. Module Initialization Contracts . . . 64
27.2. Vtable Dispatch Pattern . . . 64
27.3. Trait System (JUNO_TRAIT_ROOT) . . . 64
27.4. Design Rationale: Pointer Trait System . . . 65
27.5. Software Bus Integration Pattern . . . 65
27.6. Design Rationale: Single-Threaded Broker . . . 65
27.7. Scheduler Integration Pattern . . . 66
27.8. Design Rationale: Cyclic Executive . . . 66

1. Introduction

1.1. Purpose

This Software Design Document (SDD) describes the internal design of LibJuno version 1.0.1, a freestanding C11 embedded systems micro-framework. It follows the IEEE 1016 standard structure and is intended for developers, integrators, reviewers, and certification auditors who need to understand the design decisions, data structures, interfaces, and algorithms within the library.

1.2. Scope

This SDD covers all modules of LibJuno:

- Core infrastructure: Status, Module System, Macros, Common Types
- Memory subsystem: Pointer, Memory Allocator, Block Allocator
- Data structures: Array, Heap, Queue, Stack, Map
- Algorithms: CRC, Hash, Math (vectors, quaternions)
- System services: Time, IO (Async, I2C, SPI), Log, Mutex
- Frameworks: State Machine, Software Bus Broker, Application, Scheduler

1.3. Definitions, Acronyms, and Abbreviations

Term	Definition
DI	Dependency Injection — a design pattern where dependencies are provided to a module rather than created internally.
Vtable	Virtual function table — a struct of function pointers providing polymorphic dispatch.
Root	The common base struct of a module, containing the API pointer and failure handler fields.
Derivation	A concrete implementation struct that embeds the root as its first member.
Union	A C union of root and all derivations, sized for the largest variant.
Trait	A lightweight interface (via `JUNO_TRAIT_ROOT`) carrying only an API pointer, without failure handler fields.
Fat pointer	`JUNO_POINTER_T` — a descriptor carrying address, size, alignment, and an operations API.
Result type	A struct combining a `JUNO_STATUS_T` status code with a typed success payload.
Option type	A struct combining a boolean presence flag with a typed payload.
WCET	Worst-Case Execution Time.
MID	Message ID — a topic identifier used by the Software Bus broker.
Pipe	A queue-based subscriber channel in the Software Bus, carrying messages for a specific MID.

1.4. References

- [IEEE1016] IEEE Std 1016-2009, *IEEE Standard for Information Technology — Systems Design — Software Design Descriptions*.
- [IEEE830] IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
- [SRS] LibJuno Software Requirements Specification (generated from `requirements/` directory).
- [RTM] LibJuno Requirements Traceability Matrix (generated from code/test annotations).
- [DOXYGEN] LibJuno API Reference Documentation (generated via Doxygen).

2. System Architecture

2.1. Architectural Overview

LibJuno is a dependency-injection-oriented C11 micro-framework for embedded systems. Its

architecture is built on four foundational principles:

1. **Zero dynamic allocation** — all memory is caller-owned and injected.
2. **Freestanding compatibility** — the library compiles with `-nostdlib -ffreestanding`.
3. **Vtable-based dependency injection** — every module uses function-pointer tables for polymorphic dispatch.
4. **Explicit error propagation** — all fallible functions return status codes; no exceptions, no `errno`, no `longjmp`.

The library provides reusable building blocks (data structures, memory management, CRC, math, time, IO abstractions) and a lightweight application framework (app lifecycle, state machine, software bus, scheduler) that integrators compose into their system.

2.2. Design Rationale: Why Vtable-Based DI

LibJuno chose vtable-based dependency injection over alternative C polymorphism approaches (switch-based dispatch, `_Generic`, per-instance function pointers) for the following reasons:

Architectural decoupling. A root object carrying a `const API_T *ptApi` pointer supports passing modules behind a stable root type. Dependencies can be injected explicitly, and concrete implementations can be swapped without central registries or framework-owned dispatch code.

Why not switch-based dispatch. A switch-based approach centralizes polymorphism around a type tag, coupling the common interface layer to every concrete implementation. Each new backend requires editing the switch logic and shared dispatch functions — the opposite of LibJuno's goal of making components easy to adapt, share, and compose. With the vtable pattern, a new implementation defines a new derived type plus a new static API table, without requiring edits to a central dispatcher.

Vtable indirection vs. switch dispatch. A vtable call adds one pointer dereference and typically prevents cross-interface inlining. A switch-based dispatcher can sometimes be inlined more aggressively. LibJuno favors architectural decoupling, substitutability, and test injection over dispatch micro-optimization — consistent with its emphasis on portability, explicit interfaces, and reusable building blocks.

Shared vtable per type vs. per-instance function pointers. LibJuno stores `const API_T *ptApi` in the root — one pointer per instance, with the function table shared by all instances of the same implementation. This is smaller and more consistent than per-instance function pointers, especially for embedded targets with many instances sharing the same behavior.

`const` vtable and ROM placement. The module macros define the root field as `const API_T *ptApi`, and implementations provide `static const` API tables. This supports placing vtables in read-only storage/flash on embedded systems, reducing RAM pressure and preventing accidental mutation.

Why not `_Generic` or macro-driven generics. `_Generic` is selected at compile time from the static type at the call site; it does not provide runtime substitution through a common root pointer. LibJuno's module system is explicitly built around runtime polymorphism and injected dependencies, so `_Generic` would supplement type ergonomics, not replace the architecture.

2.3. Module System Pattern: Root / Derivation / Union

Every LibJuno module follows a three-part structural pattern:

```
// 1. Root — common interface fields
struct MY_ROOT_TAG JUNO_MODULE_ROOT(MY_API_T,
    // additional root-level state
);

// 2. Derivation — embeds root as first member
struct MY_IMPL_TAG JUNO_MODULE_DERIVE(MY_ROOT_T,
    // implementation-specific state
);

// 3. Union — sized for the largest variant
union MY_MODULE_T JUNO_MODULE(MY_API_T, MY_ROOT_T,
    MY_IMPL_T tImpl;
);
```

2.4. Design Rationale: Root/Derivation/Union Layering

Why the root/derivation split. API functions accept `ROOT_T *` — the common base type. A flat struct per implementation would require every call site to know the concrete type (breaking abstraction) or use `void *` casts (breaking type safety). Because `JUNO_MODULE_SUPER` is always the first member, a `DERIVATION_T *` can be safely cast to `ROOT_T *` by the C standard (§6.7.2.1¶15).

Why the union. The union serves stack allocation sizing with type erasure. LibJuno requires zero dynamic allocation, so the caller must allocate the module instance — but may not know which derivation will be initialized at runtime. The union is sized to the largest derivation. It also provides C-standard type-punning safety (§6.5.2.3): accessing the root member is always valid because the root is a common initial sequence shared by all members.

Why first-member embedding over `container_of`. Three reasons:

1. **No `offsetof` dependency** — the cast is `(ROOT_T *)ptDerived`, zero arithmetic, zero macros.
2. **No ownership inversion** — root and derivation are physically contiguous in a single allocation.
3. **Determinism and locality** — same cache line, no pointer indirection.

2.5. Subsystem Decomposition

LibJuno is organized into the following subsystems:

Subsystem	Directory	Modules
Core	`include/juno/`	Status, Module, Macros, Types
Memory	`include/juno/memory/`	Pointer, Memory Allocator, Block Allocator

Subsystem	Directory	Modules
Data Structures	`include/juno/ds/`	Array, Heap, Queue, Stack, Map
Algorithms	`include/juno/crc/`, `hash/`, `math/`	CRC (ARC, BinHex, CCITT, CCITT32, Kermit, ZIP), Hash (DJB2), Math (vectors, quaternions)
Time	`include/juno/time/`	Time module
IO	`include/juno/io/`	Async IO, I2C IO, SPI IO
Logging	`include/juno/log/`	Log module
Concurrency	`include/juno/mp/`	Mutex
State Machine	`include/juno/sm/`	State Machine
Software Bus	`include/juno/sb/`	Broker
Application	`include/juno/app/`	Application lifecycle
Scheduler	`include/juno/sch/`	Cyclic executive scheduler

2.6. Module Dependency Overview

The dependency graph flows from high-level frameworks down to core primitives:

```
Scheduler ──→ Application ──→ Module System
     │                              │
     ├──→ Time                      ├──→ Status
     │                              └──→ Macros
Software Bus ──→ Queue ──→ Array ──→ Pointer
     │                                  │
     │                                  └──→ Status
Map ──→ Array ──→ Pointer
     │
     ├──→ Hash
     └──→ Value Pointer API

Heap ──→ Array ──→ Pointer

Stack ──→ Array ──→ Pointer

Block Allocator ──→ Memory Allocator API ──→ Pointer

State Machine ──→ Module System

CRC, Math, Hash ──→ (freestanding, no module dependencies)

IO (Async, I2C, SPI) ──→ Module System, Time (Async only)

Log ──→ Module System

Mutex ──→ Module System
```

3. Detailed Design

Each subsection below describes one module's detailed design. Subsections are maintained as separate AsciiDoc files for modularity.

4. Status Module

4.1. Purpose

The Status module (`status.h`) defines the unified error-reporting type and failure-handling hooks used by every other module in LibJuno. It provides canonical status codes, the failure handler callback signature, and helper macros for invoking failure handlers without altering control flow.

4.2. Data Structures

`JUNO_STATUS_T` (`int32_t`)

The unified status code type. Zero (`JUNO_STATUS_SUCCESS`) indicates success; non-zero values indicate specific error conditions.

Status Code Constants

Code	Value	Meaning
`JUNO_STATUS_SUCCESS`	0	Operation completed successfully
`JUNO_STATUS_ERR`	1	Unspecified error
`JUNO_STATUS_NULLPTR_ERROR`	2	Required pointer was NULL
`JUNO_STATUS_MEMALLOC_ERROR`	3	Memory allocation failed
`JUNO_STATUS_MEMFREE_ERROR`	4	Memory free operation failed
`JUNO_STATUS_INVALID_TYPE_ERROR`	5	Type or trait mismatch
`JUNO_STATUS_INVALID_SIZE_ERROR`	6	Invalid size or alignment
`JUNO_STATUS_TABLE_FULL_ERROR`	7	Fixed-capacity table full
`JUNO_STATUS_DNE_ERROR`	8	Element or key does not exist
`JUNO_STATUS_FILE_ERROR`	9	Generic file IO error
`JUNO_STATUS_READ_ERROR`	10	Read operation failed
`JUNO_STATUS_WRITE_ERROR`	11	Write operation failed
`JUNO_STATUS_CRC_ERROR`	12	CRC check failed
`JUNO_STATUS_INVALID_REF_ERROR`	13	Invalid reference handle
`JUNO_STATUS_REF_IN_USE_ERROR`	14	Reference still active
`JUNO_STATUS_INVALID_DATA_ERROR`	15	Input data failed validation
`JUNO_STATUS_TIMEOUT_ERROR`	16	Operation timed out

Code	Value	Meaning
`JUNO_STATUS_OOB_ERROR`	17	Index or pointer out of bounds

`JUNO_FAILURE_HANDLER_T`

Callback signature: `void (*)(JUNO_STATUS_T tStatus, const char *pcCustomMessage, JUNO_USER_DATA_T *pvUserData)`. Invoked for diagnostic purposes only — never alters control flow.

`JUNO_USER_DATA_T`

Opaque `void` type for user-supplied context passed to failure handlers.

4.3. Interface Design

`JUNO_FAIL(tStatus, pfcnFailureHandler, pvFailureUserData, pcMessage)`

Macro that invokes the failure handler (if non-NULL) with the status, message, and user data. Does not alter control flow — callers must still return or handle the error.

`JUNO_FAIL_MODULE(tStatus, ptMod, pcMessage)`

Macro that invokes a module instance's failure handler via `JUNO_MODULE_SUPER` fields. Null-safe: does nothing if `ptMod` or its handler is NULL.

`JUNO_FAIL_ROOT(tStatus, ptMod, pcMessage)`

Macro that invokes a module root's failure handler directly. Null-safe: does nothing if `ptMod` or its handler is NULL.

4.4. Algorithm Descriptions

No algorithms. All operations are macro-based conditional invocations with O(1) cost.

4.5. Error Handling

The Status module itself defines the error-handling primitives used elsewhere. The `JUNO_FAIL*` macros guard against NULL before calling the handler, preventing null dereferences. The failure handler is diagnostic-only; it has no return value and does not affect the caller's control flow.

4.6. Design Rationale

Status codes are `int32_t` rather than an enum to allow extensibility beyond the pre-defined set while maintaining a fixed-size ABI across compilation units. Failure handlers are callback-based (not `abort` or `longjmp`) to ensure deterministic control flow suitable for safety-critical embedded systems.

4.7. Requirements Traceability

Requirement	Coverage
REQ-STATUS-001	`JUNO_STATUS_T` type definition
REQ-STATUS-002	Status code constants (SUCCESS through OOB_ERROR)
REQ-STATUS-003	`JUNO_FAILURE_HANDLER_T` callback signature
REQ-STATUS-004	`JUNO_FAIL` macro implementation

5. Module System

5.1. Purpose

The Module System (`module.h`) provides the structural primitives for vtable-based dependency injection in LibJuno. It defines macros for declaring module roots, derivations, and unions, as well as Result and Option wrapper types for deterministic error propagation.

5.2. Data Structures

5.2.1. Module Root (`JUNO_MODULE_ROOT`)

Expands to a struct body with fields:

```
const API_T *ptApi;                    // Vtable pointer (shared, const)
JUNO_FAILURE_HANDLER_T _pfcnFailureHandler;  // Optional failure callback
JUNO_USER_DATA_T *_pvFailureUserData;        // Opaque user data for handler
// ... caller-specified additional members
```

5.2.2. Module Derivation (`JUNO_MODULE_DERIVE`)

Expands to a struct body whose first member is the root:

```
ROOT_T tRoot;   // Embedded root (via JUNO_MODULE_SUPER)
// ... implementation-specific members
```

The first-member embedding guarantees that casting `DERIVATION_T *` to `ROOT_T *` is valid per C11 §6.7.2.1¶15.

5.2.3. Module Union (`JUNO_MODULE`)

Expands to a union containing the root and all derivations:

```
union {
    ROOT_T tRoot;           // Common view (via JUNO_MODULE_SUPER)
    DERIVATION_1_T tImpl1;  // Sized for this variant
```

```
    DERIVATION_2_T tImpl2;  // Sized for this variant
};
```

The union is sized to the largest derivation, enabling caller-side stack allocation without knowing the concrete type at runtime.

5.2.4. Trait Root (`JUNO_TRAIT_ROOT`)

A lightweight variant of `JUNO_MODULE_ROOT` that omits the failure handler fields:

```
const API_T *ptApi;
// ... trait-specific members (no failure handler)
```

Used for ephemeral, pass-by-value types like `JUNO_POINTER_T`.

5.2.5. Result Types (`JUNO_MODULE_RESULT`)

Generates a struct with fields `{JUNO_STATUS_T tStatus; OK_T tOk;}`. Pre-defined specializations in `types.h`: `JUNO_RESULT_BOOL_T`, `JUNO_RESULT_UINT32_T`, `JUNO_RESULT_F64_T`, `JUNO_RESULT_SIZE_T`, `JUNO_RESULT_VOID_PTR_T`.

5.2.6. Option Types (`JUNO_MODULE_OPTION`)

Generates a struct with fields `{bool bIsSome; SOME_T tSome;}`. Models optional presence where absence is a legitimate state.

5.3. Interface Design

5.3.1. Construction Macros

`JUNO_MODULE_DECLARE(NAME_T)`

Forward-declare a module union type.

`JUNO_MODULE_ROOT_DECLARE(NAME_T)`

Forward-declare a root struct type.

`JUNO_MODULE_DERIVE_DECLARE(NAME_T)`

Forward-declare a derivation struct type.

5.3.2. Access Macros

`JUNO_MODULE_SUPER`

Standard field name (`tRoot`) for the embedded root in derivations/unions.

`JUNO_MODULE_GET_API(ptModule, ROOT_T)`

Cast to `ROOT_T *` and retrieve `ptApi`.

5.3.3. Result/Option Macros

`JUNO_OK(result)`

Access `result.tOk`.

`JUNO_ASSERT_OK(result, …)`

Execute fallback statements if `result.tStatus != SUCCESS`.

`JUNO_OK_RESULT(value)`

Construct a success result literal.

`JUNO_ERR_RESULT(err, value)`

Construct an error result literal.

`JUNO_SOME(result)`

Access `result.tSome`.

`JUNO_ASSERT_SOME(result, …)`

Execute fallback statements if `result.bIsSome` is false.

`JUNO_SOME_OPTION(value)`

Construct a present option literal.

`JUNO_NONE_OPTION(default_value)`

Construct an absent option literal.

5.4. Algorithm Descriptions

No algorithms. The module system is purely structural — all operations are compile-time macro expansions.

5.5. Error Handling

Result types enforce check-before-use via `JUNO_ASSERT_OK`. Option types enforce presence-checking via `JUNO_ASSERT_SOME`. Both macros support variadic failure actions that the caller provides inline (typically `return status`).

5.6. Design Rationale

The module system implements the root/derivation/union pattern described in Section 2 of this SDD. The root carries the vtable pointer as its first logical field, enabling safe upcast from any derivation or union view. The union provides stack-allocation sizing with type erasure. `JUNO_TRAIT_ROOT` omits failure handler fields for lightweight, ephemeral interfaces (see Pointer trait rationale in Section 5). Result/Option types provide structural error checking without `errno`, `longjmp`, or exceptions — suitable for freestanding C11.

5.7. Requirements Traceability

Requirement	Coverage
REQ-MODULE-001	`JUNO_MODULE_ROOT` — root struct with `ptApi` field
REQ-MODULE-002	`JUNO_MODULE_ROOT` — failure handler and user data fields
REQ-MODULE-003	`JUNO_MODULE_DERIVE` — first-member root embedding
REQ-MODULE-004	`JUNO_MODULE` — union of root and derivations
REQ-MODULE-005	`JUNO_MODULE_RESULT` — Result type definition
REQ-MODULE-006	`JUNO_MODULE_OPTION` — Option type definition

6. Macros Module

6.1. Purpose

The Macros module (`macros.h`) provides lightweight, freestanding-friendly assertion macros used throughout LibJuno. They return error codes rather than aborting, supporting deterministic error propagation without exceptions or `longjmp`.

6.2. Data Structures

No standalone data structures. The macros operate on `JUNO_STATUS_T` values and module pointers defined by the Status and Module System modules.

6.3. Interface Design

`JUNO_ASSERT_EXISTS(ptr)`

Returns `JUNO_STATUS_NULLPTR_ERROR` if the expression `ptr` evaluates to false (NULL, zero). Intended for validating required pointers within functions that return `JUNO_STATUS_T`.

`JUNO_ASSERT_EXISTS_MODULE(ptr, ptMod, str)`

Like `JUNO_ASSERT_EXISTS` but also invokes the module's failure handler via `JUNO_FAIL_MODULE` before returning. Provides diagnostic context via the `str` message.

`JUNO_ASSERT_SUCCESS(tStatus, …)`

Executes the variadic statements (typically `return tStatus`) if `tStatus != JUNO_STATUS_SUCCESS`. Used for ergonomic error propagation in call chains.

6.4. Algorithm Descriptions

No algorithms. All macros are O(1) conditional checks that expand inline.

6.5. Error Handling

Each macro is itself an error-handling primitive:

- `JUNO_ASSERT_EXISTS` maps NULL pointers to `JUNO_STATUS_NULLPTR_ERROR` by construction.
- `JUNO_ASSERT_SUCCESS` propagates any non-success status using caller-supplied actions.
- `JUNO_ASSERT_EXISTS_MODULE` combines null-checking with diagnostic notification.

The macro names are self-documenting proof obligations: `ASSERT_EXISTS` reads as "this pointer must exist."

6.6. Design Rationale

The ASSERT macro family was chosen over raw `if` checks for three reasons:

1. **Correct status by construction** — each macro maps to a specific `JUNO_STATUS_T` code, preventing mismatched return values.
2. **Ergonomic variadic propagation** — `JUNO_ASSERT_SUCCESS(s, return s)` is concise and prevents forgetting error checks.
3. **Self-documenting** — reading `JUNO_ASSERT_EXISTS(ptArray)` clearly states the obligation without additional comments.

6.7. Requirements Traceability

Requirement	Coverage
REQ-SYS-007	`JUNO_ASSERT_EXISTS` — null-pointer validation
REQ-SYS-008	`JUNO_ASSERT_EXISTS` + `JUNO_ASSERT_SUCCESS` — error propagation
REQ-SYS-010	`JUNO_ASSERT_EXISTS` — precondition validation
REQ-SYS-011	`JUNO_ASSERT_EXISTS` — status return on violation

7. Common Types Module

7.1. Purpose

The Types module (`types.h`) provides convenience specializations of the generic `JUNO_MODULE_RESULT` macro for frequently used payload types. These result types are used across all LibJuno modules to return a status alongside a typed value in a single aggregate.

7.2. Data Structures

`JUNO_RESULT_BOOL_T`

```
{JUNO_STATUS_T tStatus; bool tOk;}
```

`JUNO_RESULT_UINT32_T`

`{JUNO_STATUS_T tStatus; uint32_t tOk;}`

`JUNO_RESULT_F64_T`

`{JUNO_STATUS_T tStatus; double tOk;}`

`JUNO_RESULT_SIZE_T`

`{JUNO_STATUS_T tStatus; size_t tOk;}`

`JUNO_RESULT_VOID_PTR_T`

`{JUNO_STATUS_T tStatus; void *tOk;}`

All types follow the `JUNO_MODULE_RESULT` pattern: access via `JUNO_OK(r)` and validate via `JUNO_ASSERT_OK(r, …)`.

7.3. Interface Design

No functions or macros beyond the type definitions. Types are used as return values by other modules' API functions.

7.4. Algorithm Descriptions

No algorithms.

7.5. Error Handling

Each result type embeds `JUNO_STATUS_T` for structural check-before-use. Callers use `JUNO_ASSERT_OK(result, return result.tStatus)` to validate before accessing `tOk`.

7.6. Design Rationale

Pre-defined specializations avoid repetitive `JUNO_MODULE_RESULT` invocations across modules. The set covers the most common C value types used in embedded contexts: boolean flags, 32-bit counts, double-precision floats, sizes, and opaque pointers.

7.7. Requirements Traceability

Requirement	Coverage
REQ-MODULE-005	All types defined via `JUNO_MODULE_RESULT` macro

8. Pointer Module

8.1. Purpose

The Pointer module (`pointer_api.h`) defines the fat pointer trait (`JUNO_POINTER_T`) and associated APIs used by all data structures and memory modules in LibJuno. It provides self-describing memory descriptors that carry address, size, alignment, and operations — enabling type-safe, `memcpy`-free element manipulation in freestanding environments.

8.2. Data Structures

8.2.1. `JUNO_POINTER_T` (Trait Root)

```
struct JUNO_POINTER_TAG JUNO_TRAIT_ROOT(JUNO_POINTER_API_T,
    void *pvAddr;       // Address of the memory region
    size_t zSize;       // Size in bytes
    size_t zAlignment;  // Required alignment in bytes
);
```

A pass-by-value descriptor. Uses `JUNO_TRAIT_ROOT` (no failure handler fields) for minimal size.

8.2.2. `JUNO_POINTER_API_T`

```
struct JUNO_POINTER_API_TAG {
    JUNO_STATUS_T (*Copy)(JUNO_POINTER_T tDest, const JUNO_POINTER_T tSrc);
    JUNO_STATUS_T (*Reset)(JUNO_POINTER_T tPointer);
};
```

- **Copy**: copies the value from `tSrc` to `tDest` storage.
- **Reset**: resets the pointee to a defined empty state (e.g., zero-init).

8.2.3. `JUNO_VALUE_POINTER_API_T`

```
struct JUNO_VALUE_POINTER_API_TAG {
    JUNO_RESULT_BOOL_T (*Equals)(const JUNO_POINTER_T tLeft, const JUNO_POINTER_T
tRight);
};
```

Content-based equality comparison used by associative containers (Map).

8.2.4. Result/Option Types

`JUNO_RESULT_POINTER_T`

`{JUNO_STATUS_T tStatus; JUNO_POINTER_T tOk;}`

`JUNO_OPTION_POINTER_T`

`{bool bIsSome; JUNO_POINTER_T tSome;}`

8.3. Interface Design

`JunoMemory_PointerInit(ptApi, TYPE_T, pvAddr)`

Macro that constructs a `JUNO_POINTER_T` literal from an API, C type (for `sizeof`/`alignof`), and address.

`JunoMemory_PointerVerifyType(pointer, type, tApi)`

Macro that verifies a pointer descriptor matches an expected type and API identity. Returns `JUNO_STATUS_SUCCESS` on match, `JUNO_STATUS_ERR` otherwise. Also checks alignment via `(uintptr_t)pvAddr % zAlignment == 0`.

`JUNO_ASSERT_POINTER_TYPE(tStatus, tPointer, tType, tApi)`

Macro that asserts a pointer's type and API, returning on failure.

`JunoMemory_PointerApiVerify(ptPointerApi)`

Inline function that verifies `Copy` and `Reset` are non-NULL.

`JunoMemory_ValuePointerApiVerify(ptPointerApi)`

Inline function that verifies `Equals` is non-NULL.

`JunoMemory_PointerVerify(tPointer)`

Inline function that validates API, address, and size are non-NULL/non-zero, then delegates to `JunoMemory_PointerApiVerify`.

8.4. Algorithm Descriptions

No algorithms. All operations are O(1) verification checks or literal construction.

8.5. Error Handling

- `JunoMemory_PointerVerify` returns `JUNO_STATUS_NULLPTR_ERROR` if API, address, or size is invalid.
- `JunoMemory_PointerVerifyType` returns `JUNO_STATUS_ERR` on any mismatch (size, alignment, API identity, alignment of address).
- No failure handler is invoked — the pointer is a trait root without failure handler fields.

8.6. Design Rationale

Fat pointer with vtable. `JUNO_POINTER_T` binds `{pvAddr, zSize, zAlignment, ptApi}` into a self-describing, self-validating value type. In freestanding environments, `memcpy` is not guaranteed; by placing `Copy` behind a vtable, the element knows how to copy itself — data structures never require `memcpy`.

Copy/Reset on the pointer, not per-data-structure. One Copy/Reset implementation per element

type, reused by every data structure (Array, Heap, Queue, Stack, Map). Data structures remain type-agnostic; adding a new element type requires only a new `JUNO_POINTER_API_T` implementation.

`JUNO_TRAIT_ROOT` **(no failure handler).** The pointer is a pass-by-value ephemeral descriptor, not a stateful module instance. Adding failure handler fields (+16 bytes on 64-bit) would bloat every data structure call's stack frame. Failure reporting belongs to the owning module.

8.7. Requirements Traceability

Requirement	Coverage
REQ-POINTER-001	`JUNO_POINTER_T` struct definition with addr/size/alignment
REQ-POINTER-002	`JunoMemory_PointerApiVerify` — Copy function present
REQ-POINTER-003	`JunoMemory_PointerApiVerify` — Reset function present
REQ-POINTER-004	`JunoMemory_PointerVerify` — full descriptor validation
REQ-POINTER-005	`JunoMemory_PointerVerifyType` — type/API/alignment match

9. Memory Module

9.1. Purpose

The Memory module (`memory_api.h`, `memory_block.h`) provides the abstract memory allocation interface and its fixed-size block allocator implementation. It enables deterministic, bounded-capacity memory management on caller-supplied backing buffers without dynamic allocation.

9.2. Data Structures

9.2.1. Abstract Allocator Root (`JUNO_MEMORY_ALLOC_ROOT_T`)

```
struct JUNO_MEMORY_ALLOC_ROOT_TAG JUNO_MODULE_ROOT(JUNO_MEMORY_ALLOC_API_T,
    const JUNO_POINTER_API_T *ptPointerApi;  // Dependent pointer operations
);
```

Follows the module root pattern with an additional dependency on the pointer API.

9.2.2. Allocator API (`JUNO_MEMORY_ALLOC_API_T`)

Function	Description
`Get(ptMem, zSize)`	Allocate a region of at least `zSize` bytes. Returns `JUNO_RESULT_POINTER_T`.
`Update(ptMem, ptMemory, zNewSize)`	Adjust an existing allocation's descriptor in-place.
`Put(ptMem, pvAddr)`	Free a previously allocated region. Clears descriptor on success.

9.2.3. Block Allocator Derivation (JUNO_MEMORY_ALLOC_BLOCK_T)

```
struct JUNO_MEMORY_ALLOC_BLOCK_TAG JUNO_MODULE_DERIVE(JUNO_MEMORY_ALLOC_ROOT_T,
    uint8_t *pvMemory;                        // Backing memory region
    JUNO_MEMORY_BLOCK_METADATA_T *ptMetadata; // Per-block free-list metadata
    size_t zTypeSize;                         // Element size per block
    size_t zAlignment;                        // Alignment per block
    size_t zLength;                           // Total blocks in pool
    size_t zUsed;                             // Blocks currently allocated
    size_t zFreed;                            // Entries on free stack
);
```

9.2.4. Block Metadata (JUNO_MEMORY_BLOCK_METADATA_T)

Each entry stores the address of a freed block; the free list is a stack in `ptMetadata[0..zFreed)`.

9.3. Interface Design

`JunoMemory_BlockInit(…)`

Initializes a block allocator over caller-supplied backing memory and metadata arrays. Validates all parameters (nulls, overflow of `zTypeSize * zLength`, base alignment). Wires the vtable into the root. Counters `zUsed` and `zFreed` start at 0.

`JunoMemory_BlockGetT(ptBlkRoot, type)`

Convenience macro that calls `Get` with `sizeof(type)`.

`JunoMemory_BlockPutT(ptBlkRoot, pPtr)`

Convenience macro that calls `Put` on a pointer descriptor.

`JunoMemory_AllocApiVerify(ptAllocApi)`

Inline verification that all three API functions (Get, Put, Update) are non-NULL.

`JunoMemory_AllocVerify(ptAlloc)`

Inline verification of the allocator root, its API, and its dependent pointer API.

9.3.1. Storage Declaration Macros

`JUNO_MEMORY_BLOCK(name, type, length)`

Declares a static backing array of `type[length]`.

`JUNO_MEMORY_BLOCK_METADATA(name, length)`

Declares a static metadata array of `JUNO_MEMORY_BLOCK_METADATA_T[length]`.

9.4. Algorithm Descriptions

9.4.1. Block Get (Allocation)

1. If a freed block exists (`zFreed > 0`): pop from free stack (LIFO), decrement `zFreed`.
2. Otherwise: carve a new block from offset `zUsed * zTypeSize` if `zUsed < zLength`; increment `zUsed`.
3. Reset the block via the pointer API before returning.
4. On Reset failure, roll back and report the error.

Complexity: O(1).

9.4.2. Block Put (Deallocation)

1. Verify the address belongs to the pool and is properly aligned.
2. Reject double-frees.
3. If the block is the most recently allocated (last in sequence): decrement `zUsed` directly.
4. Otherwise: push the address onto the free stack; increment `zFreed`.
5. Clear the caller's descriptor (`pvAddr = NULL`, sizes zeroed).

Complexity: O(1).

9.4.3. Block Update

1. Adjust the descriptor's `zSize` in-place, up to `zTypeSize`.
2. Reject requests exceeding `zTypeSize`. Memory is never moved.

Complexity: O(1).

9.5. Error Handling

- `Get`: returns `JUNO_STATUS_MEMALLOC_ERROR` when the pool is full or size exceeds element size. Returns `JUNO_STATUS_NULLPTR_ERROR` for zero-size requests.
- `Put`: returns `JUNO_STATUS_MEMFREE_ERROR` for unaligned, out-of-range, or double-freed addresses. Restores the descriptor on failure.
- `Update`: returns `JUNO_STATUS_MEMALLOC_ERROR` if the new size exceeds element capacity.
- All operations call `Verify` at entry.

9.6. Design Rationale

The abstract `JUNO_MEMORY_ALLOC_API_T` separates the allocation interface from the block allocator implementation, allowing future allocator strategies (slab, buddy) to be injected through the same root type. The block allocator provides O(1) deterministic operations suitable for hard real-time systems with fixed-capacity pools.

9.7. Requirements Traceability

Requirement	Coverage
REQ-MEMORY-001	`JUNO_MEMORY_ALLOC_API_T` — Get/Put/Update vtable
REQ-MEMORY-002	`JUNO_MEMORY_ALLOC_ROOT_T` — root with pointer API dependency
REQ-MEMORY-003	`JunoMemory_BlockInit` — block allocator initialization
REQ-MEMORY-004	Block Get — O(1) allocation from pool
REQ-MEMORY-005	Block Put — O(1) deallocation with free-stack
REQ-MEMORY-006	Block Update — in-place size adjustment
REQ-MEMORY-007	Block metadata tracks free list
REQ-MEMORY-008	Verification of allocator and pointer API

10. Array Module

10.1. Purpose

The Array module (`array_api.h`) provides a minimal, fixed-capacity array abstraction used as the foundational storage layer by higher-level data structures (Heap, Queue, Stack, Map). It defines three core operations — SetAt, GetAt, RemoveAt — over caller-owned, contiguous storage.

10.2. Data Structures

10.2.1. Array Root (`JUNO_DS_ARRAY_ROOT_T`)

```
struct JUNO_DS_ARRAY_ROOT_TAG JUNO_MODULE_ROOT(JUNO_DS_ARRAY_API_T,
    size_t zCapacity;  // Fixed capacity defined at initialization
);
```

10.2.2. Array API (`JUNO_DS_ARRAY_API_T`)

Function	Description
`SetAt(ptArray, tItem, iIndex)`	Copy `tItem` into slot `iIndex` via the pointer API.
`GetAt(ptArray, iIndex)`	Return a `JUNO_RESULT_POINTER_T` describing the slot at `iIndex`.
`RemoveAt(ptArray, iIndex)`	Reset the slot at `iIndex` via the pointer API.

10.3. Interface Design

`JunoDs_ArrayInit(ptArray, ptArrayApi, iCapacity, pfcnFailureHdlr, pvUserData)`

Wires the vtable, sets capacity, and calls `JunoDs_ArrayVerify` to validate the configuration. Returns `JUNO_STATUS_NULLPTR_ERROR` if any required parameter is NULL or zero.

`JunoDs_ArrayVerify(ptArray)`

Inline verification: checks ptArray non-NULL, capacity non-zero, and all API function pointers present.

`JunoDs_ArrayApiVerify(ptArrayApi)`

Inline verification of the vtable entries.

`JunoDs_ArrayVerifyIndex(ptArray, iIndex)`

Validates index within capacity. Returns `JUNO_STATUS_OOB_ERROR` and invokes failure handler on out-of-bounds.

10.4. Algorithm Descriptions

All operations are O(1) index-based access into contiguous storage. No sorting, searching, or rebalancing is performed at this layer.

10.5. Error Handling

- `SetAt`: returns `JUNO_STATUS_OOB_ERROR` for invalid indices; pointer errors from Copy.
- `GetAt`: returns `JUNO_STATUS_OOB_ERROR` for invalid indices.
- `RemoveAt`: returns `JUNO_STATUS_OOB_ERROR` for invalid indices; pointer errors from Reset.
- All functions call `JunoDs_ArrayVerify` at entry.
- `JunoDs_ArrayVerifyIndex` invokes the root's failure handler on OOB.

10.6. Design Rationale

The Array module is deliberately thin — it provides only indexed access without length tracking, ordering, or growth semantics. Higher-level data structures (Heap, Queue, Stack, Map) compose the Array for storage and add their own invariants. This separation keeps the Array reusable and avoids duplicating contiguous-storage logic across multiple modules.

10.7. Requirements Traceability

Requirement	Coverage
REQ-ARRAY-001	`JunoDs_ArrayInit` — initialization with capacity and API
REQ-ARRAY-002	`SetAt` — copy element into indexed slot
REQ-ARRAY-003	`GetAt` — retrieve pointer descriptor for indexed slot
REQ-ARRAY-004	`RemoveAt` — reset indexed slot

Requirement	Coverage
REQ-ARRAY-005	`JunoDs_ArrayVerifyIndex` — bounds checking
REQ-ARRAY-006	`JunoDs_ArrayVerify` — capacity validation
REQ-ARRAY-007	`JunoDs_ArrayVerify` — API completeness check

11. Heap Module

11.1. Purpose

The Heap module (`heap_api.h`) provides a binary heap built on the Array abstraction. It supports Insert, Pop, Heapify, Update, and SiftDown operations, parameterized by user-supplied Compare and Swap callbacks. The heap can be configured as a max-heap or min-heap depending on the Compare implementation.

11.2. Data Structures

11.2.1. Heap Root (`JUNO_DS_HEAP_ROOT_T`)

```
struct JUNO_DS_HEAP_ROOT_TAG JUNO_MODULE_ROOT(JUNO_DS_HEAP_API_T,
    const JUNO_DS_HEAP_POINTER_API_T *ptHeapPointerApi;  // Compare/Swap callbacks
    JUNO_DS_ARRAY_ROOT_T *ptHeapArray;                    // Backing array
    size_t zLength;                                       // Current element count
);
```

11.2.2. Heap Pointer API (`JUNO_DS_HEAP_POINTER_API_T`)

Function	Description
`Compare(ptHeap, tParent, tChild)`	Returns `true` if heap property holds for (parent, child).
`Swap(ptHeap, tLeft, tRight)`	Exchanges two elements in the backing storage.

11.2.3. Heap API (`JUNO_DS_HEAP_API_T`)

Function	Description
`Insert(ptHeap, tValue)`	Copy element, increment length, bubble up.
`Heapify(ptHeap)`	Transform backing array into a heap in-place.
`Pop(ptHeap, tReturn)`	Copy root to tReturn, swap with last, decrement, sift down.

11.2.4. Helper Result/Option Types

`JUNO_DS_HEAP_INDEX_RESULT_T`, `JUNO_DS_HEAP_INDEX_OPTION_T`, `JUNO_DS_HEAP_INDEX_OPTION_RESULT_T`,

`JUNO_DS_HEAP_COMPARE_RESULT_T` — internal types for index arithmetic and comparison results.

11.3. Interface Design

`JunoDs_Heap_Init(ptHeap, ptHeapPointerApi, ptHeapArray, pfcnFailureHdlr, pvUserData)`

Wires the heap vtable, stores dependencies, and calls `JunoDs_Heap_Verify`.

`JunoDs_Heap_Insert(ptHeap, tValue)`

O(log n) insert with bubble-up.

`JunoDs_Heap_Pop(ptHeap, tReturn)`

O(log n) pop with sift-down.

`JunoDs_Heap_Heapify(ptHeap)`

O(n) in-place heapification via repeated sift-down from the last parent.

`JunoDs_Heap_Update(ptHeap)`

O(log n) bubble-up of the last element.

`JunoDs_Heap_SiftDown(ptHeap, iStart)`

O(log n) sift-down from a starting index.

`JunoDs_Heap_Verify(ptHeap)`

Inline function verifying heap root, array, pointer API (Compare/Swap), and all heap API function pointers. Delegates to `JunoDs_ArrayVerify` for the backing array.

11.4. Algorithm Descriptions

11.4.1. Insert

1. Copy `tValue` into slot `zLength` via `SetAt`.
2. Increment `zLength`.
3. Bubble up: compare element with parent; swap if heap property violated; repeat until root or property holds.

11.4.2. Pop

1. Copy root element (index 0) into `tReturn` via `Copy`.
2. Swap root with last element (index `zLength - 1`).
3. Remove last slot; decrement `zLength`.
4. Sift down from index 0.

11.4.3. Heapify

1. For `i` from `(zLength / 2 - 1)` down to 0: call `SiftDown(i)`.

2. This produces a valid heap in O(n) by bottom-up construction.

11.4.4. SiftDown

1. Starting at `iStart`, compare with left and right children.
2. Swap with the child that violates the heap property (if any).
3. Repeat until no swap or leaf reached.

11.5. Error Handling

- `Insert` returns `JUNO_STATUS_ERR` when the heap is full (`zLength == capacity`).
- `Pop` returns `JUNO_STATUS_ERR` when empty (`zLength == 0`).
- `Heapify` returns `JUNO_STATUS_ERR` when empty.
- All operations propagate errors from Compare, Swap, SetAt, GetAt, and RemoveAt.
- `Verify` is called at the start of every public function.

11.6. Design Rationale

The heap is parameterized by Compare/Swap rather than hardcoding a comparison operator, allowing the same heap implementation to serve max-heaps, min-heaps, and priority queues with arbitrary key types. The backing Array provides storage; the Pointer API provides element copying — keeping the heap logic type-agnostic.

11.7. Requirements Traceability

Requirement	Coverage
REQ-HEAP-001	`JunoDs_Heap_Init` — initialization with array and pointer API
REQ-HEAP-002	`Insert` — O(log n) insertion
REQ-HEAP-003	`Pop` — O(log n) root extraction
REQ-HEAP-004	`Heapify` — O(n) in-place construction
REQ-HEAP-005	`SiftDown` — O(log n) downward restoration
REQ-HEAP-006	`Update` — O(log n) upward restoration
REQ-HEAP-007	`JunoDs_Heap_Verify` — configuration validation
REQ-HEAP-008	`JunoDs_Heap_Verify` — API completeness check

12. Queue Module

12.1. Purpose

The Queue module (`queue_api.h`) provides a fixed-capacity FIFO queue built on the Array

abstraction. Elements are stored in a circular buffer pattern with O(1) enqueue, dequeue, and peek operations.

12.2. Data Structures

12.2.1. Queue Root (JUNO_DS_QUEUE_ROOT_T)

```
struct JUNO_DS_QUEUE_ROOT_TAG JUNO_MODULE_ROOT(JUNO_DS_QUEUE_API_T,
    JUNO_DS_ARRAY_ROOT_T *ptQueueArray;  // Backing array
    size_t iStartIndex;                   // Index of logical front
    size_t zLength;                       // Current element count
);
```

12.2.2. Queue API (JUNO_DS_QUEUE_API_T)

Function	Description
`Enqueue(ptQueue, tItem)`	Copy element into the back slot.
`Dequeue(ptQueue, tReturn)`	Copy front element out, advance front.
`Peek(ptQueue)`	Return pointer descriptor to the front element without removal.

12.3. Interface Design

`JunoDs_QueueInit(ptQueue, ptQueueArray, pfcnFailureHdlr, pvFailureUserData)`

Wires the queue vtable, stores the backing array, and validates via `JunoDs_QueueVerify`. `iStartIndex` and `zLength` are initialized to 0.

`JunoDs_QueuePush(ptQueue, tItem)`

Implementation of `Enqueue`.

`JunoDs_QueuePop(ptQueue, tReturn)`

Implementation of `Dequeue`.

`JunoDs_QueuePeek(ptQueue)`

Implementation of `Peek`.

`JunoDs_QueueVerify(ptQueue)`

Inline verification: checks queue root, delegates to `JunoDs_ArrayVerify` for the backing array, then verifies the queue API.

`JunoDs_QueueApiVerify(ptQueueApi)`

Inline verification that Enqueue, Dequeue, and Peek are non-NULL.

12.4. Algorithm Descriptions

12.4.1. Enqueue

1. Compute back index: `(iStartIndex + zLength) % capacity`.
2. Copy `tItem` into the backing array at that index via `SetAt`.
3. Increment `zLength`.

12.4.2. Dequeue

1. Copy the element at `iStartIndex` into `tReturn` via the pointer API.
2. Remove the element at `iStartIndex` via `RemoveAt`.
3. Advance `iStartIndex = (iStartIndex + 1) % capacity`.
4. Decrement `zLength`.

12.4.3. Peek

1. Return the pointer descriptor at `iStartIndex` via `GetAt`.

All operations are O(1).

12.5. Error Handling

- `Enqueue` returns `JUNO_STATUS_INVALID_SIZE_ERROR` when full (`zLength == capacity`).
- `Dequeue` returns `JUNO_STATUS_ERR` when empty (`zLength == 0`).
- `Peek` returns `JUNO_STATUS_INVALID_SIZE_ERROR` when empty.
- All operations call `Verify` at entry and propagate array/pointer errors.

12.6. Design Rationale

The Queue reuses the Array abstraction for storage, adding only circular-index arithmetic. This avoids duplicating contiguous-storage logic and benefits from the Array's existing verification and pointer-based element access.

12.7. Requirements Traceability

Requirement	Coverage
REQ-QUEUE-001	`JunoDs_QueueVerify` — queue invariant validation
REQ-QUEUE-002	`JunoDs_QueueInit` — initialization with backing array
REQ-QUEUE-003	`Enqueue` — FIFO insertion
REQ-QUEUE-004	`Dequeue` — FIFO extraction

Requirement	Coverage
REQ-QUEUE-005	`Peek` — non-destructive front access
REQ-QUEUE-006	Circular-buffer index arithmetic

13. Stack Module

13.1. Purpose

The Stack module (`stack_api.h`) provides a fixed-capacity LIFO stack built on the Array abstraction. It supports O(1) push, pop, and peek operations.

13.2. Data Structures

13.2.1. Stack Root (`JUNO_DS_STACK_ROOT_T`)

```
struct JUNO_DS_STACK_ROOT_TAG JUNO_MODULE_ROOT(JUNO_DS_STACK_API_T,
    JUNO_DS_ARRAY_ROOT_T *ptStackArray;  // Backing array
    size_t zLength;                       // Current element count
);
```

13.2.2. Stack API (`JUNO_DS_STACK_API_T`)

Function	Description
`Push(ptStack, tItem)`	Copy element onto the top.
`Pop(ptStack, tReturn)`	Copy top element out and remove.
`Peek(ptStack)`	Return pointer descriptor to the top element.

13.3. Interface Design

`JunoDs_StackInit(ptStack, ptStackArray, pfcnFailureHdlr, pvFailureUserData)`

Wires the stack vtable, stores the backing array, and validates via `JunoDs_StackVerify`. `zLength` is initialized to 0.

`JunoDs_StackPush(ptStack, tItem)`

Implementation of `Push`.

`JunoDs_StackPop(ptStack, tReturn)`

Implementation of `Pop`.

`JunoDs_StackPeek(ptStack)`

Implementation of `Peek`.

`JunoDs_StackVerify(ptStack)`

Inline verification: checks stack root, backing array pointer, all API function pointers, and delegates to `JunoDs_ArrayVerify`.

13.4. Algorithm Descriptions

13.4.1. Push

1. Copy `tItem` into the backing array at index `zLength` via `SetAt`.
2. Increment `zLength`.

13.4.2. Pop

1. Copy the element at index `zLength - 1` into `tReturn` via the pointer API.
2. Remove the element at index `zLength - 1` via `RemoveAt`.
3. Decrement `zLength`.

13.4.3. Peek

1. Return the pointer descriptor at index `zLength - 1` via `GetAt`.

All operations are O(1).

13.5. Error Handling

- `Push` returns `JUNO_STATUS_INVALID_SIZE_ERROR` when full (`zLength == capacity`).
- `Pop` returns `JUNO_STATUS_INVALID_SIZE_ERROR` when empty (`zLength == 0`).
- `Peek` returns `JUNO_STATUS_INVALID_SIZE_ERROR` when empty.
- All operations call `Verify` at entry and propagate array/pointer errors.

13.6. Design Rationale

Like the Queue, the Stack reuses the Array for storage. The top-of-stack is always the last used index, so no circular arithmetic is needed — just a simple length counter.

13.7. Requirements Traceability

Requirement	Coverage
REQ-STACK-001	`JunoDs_StackInit` — initialization with backing array
REQ-STACK-002	`Push` — LIFO insertion
REQ-STACK-003	`Pop` — LIFO extraction
REQ-STACK-004	`Peek` — non-destructive top access

Requirement	Coverage
REQ-STACK-005	`JunoDs_StackVerify` — stack invariant validation

14. Map Module

14.1. Purpose

The Map module (`map_api.h`) provides an open-addressed hash map built on the Array and Pointer traits. It uses linear probing over a caller-supplied array and is parameterized by hash, equality, and null-detection callbacks.

14.2. Data Structures

14.2.1. Map Root (`JUNO_MAP_ROOT_T`)

```
struct JUNO_MAP_ROOT_TAG JUNO_MODULE_ROOT(JUNO_MAP_API_T,
    const JUNO_MAP_HASHABLE_POINTER_API_T *ptHashablePointerApi;  // Hash + null-check
    const JUNO_VALUE_POINTER_API_T *ptValuePointerApi;             // Key equality
    JUNO_DS_ARRAY_ROOT_T *ptHashMap;                               // Backing array
);
```

14.2.2. Hashable Pointer API (`JUNO_MAP_HASHABLE_POINTER_API_T`)

Function	Description
`Hash(tItem)`	Compute a `size_t` hash for the key.
`IsValueNull(tItem)`	Return `true` if the slot is logically empty.

14.2.3. Map API (`JUNO_MAP_API_T`)

Function	Description
`Get(ptJunoMap, tReturnItem)`	Look up by key; return pointer to matching entry.
`Set(ptJunoMap, tItem)`	Insert or overwrite an entry.
`Remove(ptJunoMap, tKey)`	Remove an entry by key.

14.3. Interface Design

`JunoDs_MapInit(ptMapRoot, ptHashablePointerApi, ptValuePointerApi, ptArray, pfcnFailureHandler, pvUserData)`

Wires the map vtable, stores all three dependency APIs and the backing array, then validates via `JunoDs_MapVerify`.

`JunoDs_MapGet(ptJunoMap, tItem)`
: Linear-probe lookup by key.

`JunoDs_MapSet(ptJunoMap, tItem)`
: Linear-probe insert or overwrite.

`JunoDs_MapRemove(ptJunoMap, tKey)`
: Linear-probe removal.

`JunoDs_MapVerify(ptMap)`
: Inline verification: checks all API pointers (Hash, IsValueNull, Equals, Set, Get, Remove), backing array, and delegates to `JunoDs_ArrayVerify`.

14.4. Algorithm Descriptions

14.4.1. Set (Insert/Update)

1. Compute `index = Hash(key) % capacity`.
2. Probe linearly: if slot is empty (`IsValueNull` returns true), copy entry via `SetAt`.
3. If slot contains a matching key (`Equals` returns true), overwrite via `SetAt`.
4. Continue probing with wrap-around until an empty slot or match is found.
5. Return `JUNO_STATUS_TABLE_FULL_ERROR` if the probe traverses the full table.

14.4.2. Get (Lookup)

1. Compute `index = Hash(key) % capacity`.
2. Probe linearly: if slot matches key (`Equals`), return pointer descriptor via `GetAt`.
3. If slot is empty (`IsValueNull`), key does not exist — return `JUNO_STATUS_DNE_ERROR`.
4. If full traversal without match, return `JUNO_STATUS_TABLE_FULL_ERROR`.

14.4.3. Remove

1. Compute `index = Hash(key) % capacity`.
2. Probe linearly: if slot matches key, reset via `RemoveAt` and return success.
3. If empty slot encountered before match, key was not present — return success (no-op).

All operations are O(1) average, O(n) worst-case under high load or clustered collisions.

14.5. Error Handling

- `Get` returns `JUNO_STATUS_DNE_ERROR` when key not found.
- `Set` returns `JUNO_STATUS_TABLE_FULL_ERROR` when no empty slot available.
- Pointer type mismatches return `JUNO_STATUS_INVALID_TYPE_ERROR`.

- All operations propagate errors from hashing, equality checks, and array access.
- All operations call `Verify` at entry.

14.6. Design Rationale

Open addressing with linear probing was chosen for cache-friendliness and zero-allocation operation over a single contiguous array. The map is parameterized by three callback APIs (hash, equality, null-check) rather than hard-coding a specific key type, keeping it type-agnostic and reusable.

14.7. Requirements Traceability

Requirement	Coverage
REQ-MAP-001	`JunoDs_MapInit` — initialization with hash/equality APIs
REQ-MAP-002	`Set` — insert or overwrite via linear probing
REQ-MAP-003	`Get` — lookup via linear probing
REQ-MAP-004	`Remove` — deletion via linear probing
REQ-MAP-005	Hash function integration (Hash callback)
REQ-MAP-006	Key equality integration (Equals callback)
REQ-MAP-007	Null-slot detection (IsValueNull callback)
REQ-MAP-008	Linear probing with wrap-around
REQ-MAP-009	`JunoDs_MapVerify` — configuration validation
REQ-MAP-010	`JunoDs_MapVerify` — API completeness check

15. CRC Module

15.1. Purpose

The CRC module (`crc.h`) provides streaming update functions for six common CRC variants: ARC, BinHex, CCITT (16-bit), CCITT32 (32-bit), Kermit, and ZIP (CRC-32). All functions are pure, reentrant, and operate without module state or dynamic allocation.

15.2. Data Structures

No structs or module types. The CRC module is a set of pure functions operating on integer CRC accumulators and byte buffers.

15.2.1. Seed Constants

Constant	Value	Variant
`JUNO_ARC_CRC_INIT`	`0`	CRC-16/ARC (IBM-16)
`JUNO_BINHEX_CRC_INIT`	`0`	BinHex 16-bit
`JUNO_CCITT_CRC_INIT`	`-1` (0xFFFF)	CRC-16/CCITT
`JUNO_CCITT32_CRC_INIT`	`-1` (0xFFFFFFFF)	CRC-32/CCITT
`JUNO_KERMIT_CRC_INIT`	`0`	CRC-16/Kermit
`JUNO_ZIP_CRC_INIT`	`-1` (0xFFFFFFFF)	CRC-32/ZIP (PKZIP)

15.3. Interface Design

All functions follow the same streaming pattern:

```
uint16_t Juno_CrcArcUpdate(uint16_t iCrc, const void *pcData, size_t zDataSize);
uint16_t Juno_CrcBinhexUpdate(uint16_t iCrc, const void *pcData, size_t zDataSize);
uint16_t Juno_CrcCcittUpdate(uint16_t iCrc, const void *pcData, size_t zDataSize);
uint32_t Juno_CrcCcitt32Update(uint32_t iCrc, const void *pcData, size_t zDataSize);
uint16_t Juno_CrcKermitUpdate(uint16_t iCrc, const void *pcData, size_t zDataSize);
uint32_t Juno_CrcZipUpdate(uint32_t iCrc, const void *pcData, size_t zDataSize);
```

- **Input**: running CRC (or seed for first chunk), data buffer, buffer length.
- **Output**: updated CRC value.
- **NULL/zero**: returns 0 when `pcData` is NULL or `zDataSize` is 0.

15.3.1. ZIP Streaming Note

`Juno_CrcZipUpdate` applies a final XOR (`0xFFFFFFFF`) before returning. For multi-chunk streaming, undo the XOR before passing it back: `crc = Juno_CrcZipUpdate(crc ^ 0xFFFFFFFF, chunk, len)`.

15.4. Algorithm Descriptions

All CRC functions use table-driven computation:

1. For each byte in the input buffer:
   a. Index into a 256-entry lookup table using the current CRC and the data byte.
   b. Update the running CRC with the table value and shift operations.
2. Return the final CRC (with final XOR for ZIP).

The lookup tables are compiled into the library's `.c` source files (`juno_arc.c`, `juno_binhex.c`, `juno_ccitt.c`, `juno_ccitt32.c`, `juno_kermit.c`, `juno_zip.c`).

Complexity: O(n) where n = `zDataSize`.

15.5. Error Handling

- NULL data pointer or zero size: returns 0 (intentional, not an error status).
- No status codes returned — CRC functions return the CRC value directly.
- Functions are pure and reentrant — no global state.

15.6. Design Rationale

CRC functions are standalone (no module root or vtable) because they are stateless, pure computations with no dependencies to inject. The streaming update pattern allows incremental CRC calculation over multiple chunks without buffering the full message. Multiple CRC variants are provided because embedded protocols commonly require specific polynomials (CCITT for XMODEM, ZIP for file integrity, Kermit for serial protocols).

15.7. Requirements Traceability

Requirement	Coverage
REQ-CRC-001	ARC CRC-16 update function
REQ-CRC-002	BinHex CRC-16 update function
REQ-CRC-003	CCITT CRC-16 update function
REQ-CRC-004	CCITT32 CRC-32 update function
REQ-CRC-005	Kermit CRC-16 update function
REQ-CRC-006	ZIP CRC-32 update function
REQ-CRC-007	Table-driven CRC computation
REQ-CRC-008	Streaming multi-chunk CRC support

16. Hash Module

16.1. Purpose

The Hash module (`hash_djb2.h`) provides the djb2 hash function, a simple and effective hash algorithm used by the Map module for key hashing.

16.2. Data Structures

No module types. The function operates on byte buffers and returns `JUNO_RESULT_SIZE_T`.

16.3. Interface Design

`JunoHash_Djb2(pcBuff, zBuffSize)`

Inline function that computes the djb2 hash over `zBuffSize` bytes starting at `pcBuff`.

- Returns `{JUNO_STATUS_SUCCESS, hash_value}` on success.
- Returns `{JUNO_STATUS_NULLPTR_ERROR, 0}` if `pcBuff` is NULL.

16.4. Algorithm Descriptions

16.4.1. djb2

```
size_t zHash = 5381;
for (size_t i = 0; i < zBuffSize; i++) {
    zHash = ((zHash << 5) + zHash) + pcBuff[i];  // zHash * 33 + byte
}
```

The recurrence `h = h * 33 + x` with seed 5381 was proposed by Daniel J. Bernstein. It provides good distribution for typical key data (strings, byte sequences) with minimal computation.

Complexity: O(n) where n = `zBuffSize`.

16.5. Error Handling

- NULL buffer: returns `JUNO_STATUS_NULLPTR_ERROR` without accessing memory.
- Zero-length buffer: returns the seed value (5381) as the hash.

16.6. Design Rationale

djb2 was selected for its simplicity, small code size, and adequate distribution for hash-table usage in embedded systems. The function is `static inline` in the header to avoid link-time overhead and enable inlining by the compiler. It returns a `JUNO_RESULT_SIZE_T` (rather than a raw `size_t`) to maintain the library's structural error-checking convention.

16.7. Requirements Traceability

Requirement	Coverage
REQ-HASH-001	`JunoHash_Djb2` — djb2 hash computation
REQ-HASH-002	NULL-pointer error handling
REQ-HASH-003	Result type with status code

17. Math Module

17.1. Purpose

The Math module (`juno_math.h`, `juno_vec.h`, `juno_vec_types.h`, `juno_math_types.h`) provides generic mathematical operations, vector algebra, and quaternion types for embedded applications. It uses

macro-based generics and compile-time type selection for freestanding-compatible math.

17.2. Data Structures

17.2.1. Scalar Types

`JUNO_MATH_F32_T` (`float`)

Single-precision floating-point.

`JUNO_MATH_F64_T` (`double`)

Double-precision floating-point.

17.2.2. Vector Types

`JUNO_MATH_VEC2_F32_T`, `JUNO_MATH_VEC2_F64_T`

2D vectors `{x, y}`.

`JUNO_MATH_VEC3_F32_T`, `JUNO_MATH_VEC3_F64_T`

3D vectors `{x, y, z}`.

`JUNO_MATH_VEC4_F32_T`, `JUNO_MATH_VEC4_F64_T`

4D vectors `{x, y, z, w}`.

17.2.3. Quaternion Types

`JUNO_MATH_QUAT_F32_T`, `JUNO_MATH_QUAT_F64_T`

Quaternions `{x, y, z, w}`.

17.2.4. Result Types

`JUNO_MATH_F32_RESULT_T`, `JUNO_MATH_F64_RESULT_T`

Scalar results.

`JUNO_MATH_VEC*_RESULT_T`

Vector results for each dimension and precision.

17.3. Interface Design

17.3.1. Generic Math Macros

`JunoMath_Abs(a)`

Absolute value.

`JunoMath_Max(a, b)`

Maximum of two values.

`JunoMath_Min(a, b)`
: Minimum of two values.

`JunoMath_Clamp(x, lo, hi)`
: Clamp value to range.

17.3.2. Vector Operations

`JunoMath_VecAdd(tVecA, tVecB)`
: Component-wise vector addition.

`JunoMath_VecSub(tVecA, tVecB)`
: Component-wise vector subtraction.

`JunoMath_VecScale(tVec, fScalar)`
: Scalar multiplication.

`JunoMath_VecDot(tVecA, tVecB)`
: Dot product.

`JunoMath_VecCross(tVecA, tVecB)`
: Cross product (3D only).

`JunoMath_VecNorm(tVec)`
: Euclidean norm (magnitude).

`JunoMath_VecNormalize(tVec)`
: Unit vector normalization.

Operations are implemented as `_Generic`-dispatched inline functions selecting the appropriate precision (F32 or F64) and dimension (Vec2, Vec3, Vec4) variant at compile time.

17.4. Algorithm Descriptions

17.4.1. Dot Product

Sum of component-wise products: $\vec{a} \cdot \vec{b} = \sum_{i} a_i \cdot b_i$.

17.4.2. Cross Product (3D)

$\vec{a} \times \vec{b} = (a_y b_z - a_z b_y,\ a_z b_x - a_x b_z,\ a_x b_y - a_y b_x)$

17.4.3. Norm

$\|\vec{v}\| = \sqrt{\sum_{i} v_i^2}$

Uses `sqrtf` (F32) or `sqrt` (F64).

17.4.4. Normalize

$\hat{v} = \vec{v} / \|\vec{v}\|$

Returns error if norm is zero.

All operations are O(d) where d is the vector dimension (2, 3, or 4).

17.5. Error Handling

- Division by zero in normalization: returns `JUNO_STATUS_ERR` with a zero vector.
- All vector operations return Result types; callers use `JUNO_ASSERT_OK` to validate.

17.6. Design Rationale

The math module uses C11 `_Generic` for compile-time type dispatch because math operations are purely value-based — there is no runtime polymorphism or state injection needed. This is the one place where `_Generic` is appropriate in LibJuno: it provides type safety and ergonomic dispatch without needing vtables. The macro-based generic approach keeps the API consistent across precisions and dimensions.

17.7. Requirements Traceability

Requirement	Coverage
REQ-MATH-001	Generic math macros (Abs, Max, Min, Clamp)
REQ-MATH-002	Vector types (2D, 3D, 4D in F32/F64)
REQ-MATH-003	Vector arithmetic (Add, Sub, Scale)
REQ-MATH-004	Dot product
REQ-MATH-005	Cross product (3D)
REQ-MATH-006	Norm and normalize
REQ-MATH-007	Quaternion types
REQ-MATH-008	Result type wrappers for all operations

18. Time Module

18.1. Purpose

The Time module (`time_api.h`) defines the time interface and provides reference implementations for time math (add, subtract) and conversions (timestamp nanos/micros/millis/double). Platform-specific operations (Now, Sleep, SleepTo) are injected via the vtable.

18.2. Data Structures

18.2.1. Timestamp (JUNO_TIMESTAMP_T)

```
struct JUNO_TIMESTAMP_TAG {
    JUNO_TIME_SECONDS_T iSeconds;       // uint32_t whole seconds
    JUNO_TIME_SUBSECONDS_T iSubSeconds; // uint32_t fixed-point fraction
};
```

The fractional field represents `(double)iSubSeconds / giSUBSECS_MAX` seconds, where `giSUBSECS_MAX` is the maximum value of `JUNO_TIME_SUBSECONDS_T` (0xFFFFFFFF).

18.2.2. Time Root (JUNO_TIME_ROOT_T)

```
struct JUNO_TIME_ROOT_TAG JUNO_MODULE_ROOT(JUNO_TIME_API_T, JUNO_MODULE_EMPTY);
```

18.2.3. Time API (JUNO_TIME_API_T)

Function	Description
`Now(ptTime)`	Get current time **(platform-injected)**.
`AddTime(ptTime, ptRetTime, tTimeToAdd)`	In-place timestamp addition.
`SubtractTime(ptTime, ptRetTime, tTimeToSubtract)`	In-place timestamp subtraction.
`SleepTo(ptTime, tTimeToWakeup)`	Sleep until a specific time **(platform-injected)**.
`Sleep(ptTime, tDuration)`	Sleep for a duration **(platform-injected)**.
`TimestampToNanos / Micros / Millis`	Convert timestamp to integer durations.
`NanosToTimestamp / MicrosToTimestamp / MillisToTimestamp`	Convert integer durations to timestamp.
`TimestampToDouble / DoubleToTimestamp`	Convert between timestamp and `double`.

18.2.4. Result Types

`JUNO_TIMESTAMP_RESULT_T`, `JUNO_TIME_SECONDS_RESULT_T`, `JUNO_TIME_MILLIS_RESULT_T`, `JUNO_TIME_MICROS_RESULT_T`, `JUNO_TIME_NANOS_RESULT_T`, `JUNO_TIME_SUBSECONDS_RESULT_T`.

18.3. Interface Design

`JunoTime_TimeApiInit(Now, SleepTo, Sleep)`

Macro that constructs a `JUNO_TIME_API_T` literal with user-supplied platform functions and LibJuno's reference math/conversion implementations.

18.3.1. LibJuno-Provided Functions

`JunoTime_AddTime(ptTime, ptRetTime, tTimeToAdd)`

Add duration to timestamp.

`JunoTime_SubtractTime(ptTime, ptRetTime, tTimeToSubtract)`

Subtract duration from timestamp.

`JunoTime_TimestampToNanos/Micros/Millis(ptTime, tTime)`

Convert timestamp to integer duration.

`JunoTime_NanosToTimestamp/MicrosToTimestamp/MillisToTimestamp(ptTime, iDuration)`

Convert integer duration to timestamp.

`JunoTime_TimestampToDouble(ptTime, tTimestamp)`

Convert timestamp to `double`.

`JunoTime_DoubleToTimestamp(ptTime, dTimestamp)`

Convert `double` to timestamp.

18.4. Algorithm Descriptions

18.4.1. AddTime

1. Add subseconds: `carry = (result overflows uint32_t)`.
2. Add seconds with carry.
3. No overflow detection on seconds (wraps at UINT32_MAX).

18.4.2. SubtractTime

1. If subtraction would underflow: saturate to `{0, 0}`, return `JUNO_STATUS_INVALID_DATA_ERROR`.
2. Otherwise: subtract subseconds with borrow, subtract seconds.

18.4.3. TimestampToNanos

1. Compute `iSeconds * 1e9 + (iSubSeconds * 1e9) / giSUBSECS_MAX`.
2. Round to nearest (half-up) for the fractional component.
3. Detect overflow on the `uint64_t` result.

18.4.4. DoubleToTimestamp

1. Integer part → `iSeconds`.
2. Fractional part → `iSubSeconds = (uint32_t)(frac * giSUBSECS_MAX)`.
3. If `frac >= 1 - 1/giSUBSECS_MAX`: round up to next whole second.

All operations are O(1).

18.5. Error Handling

- `SubtractTime`: returns `JUNO_STATUS_INVALID_DATA_ERROR` and saturates to zero on underflow.
- Integer conversions: return `JUNO_STATUS_INVALID_DATA_ERROR` on overflow.
- All functions validate the module pointer via `JUNO_ASSERT_EXISTS`.

18.6. Design Rationale

The time module splits platform-specific operations (Now, Sleep, SleepTo) from universal time math (add, subtract, conversions) via DI. Users inject their hardware clock through `JunoTime_TimeApiInit` and get all math for free. This follows the same DI principle: inject what varies, provide what's universal.

The fixed-point subsecond representation was chosen over floating-point for deterministic arithmetic on targets without an FPU. The full-scale divisor (`giSUBSECS_MAX`) provides maximum fractional resolution within a 32-bit field.

18.7. Requirements Traceability

Requirement	Coverage
REQ-TIME-001	`JUNO_TIMESTAMP_T` — timestamp struct with seconds/subseconds
REQ-TIME-002	`JUNO_TIME_API_T` — time vtable definition
REQ-TIME-003	`Now` — current time (platform-injected)
REQ-TIME-004	`AddTime` — timestamp addition
REQ-TIME-005	`SubtractTime` — timestamp subtraction with saturation
REQ-TIME-006	`SleepTo` — sleep until time (platform-injected)
REQ-TIME-007	`Sleep` — sleep for duration (platform-injected)
REQ-TIME-008	`TimestampToNanos` — conversion to nanoseconds
REQ-TIME-009	`TimestampToMicros` — conversion to microseconds
REQ-TIME-010	`TimestampToMillis` — conversion to milliseconds
REQ-TIME-011	`NanosToTimestamp` — conversion from nanoseconds
REQ-TIME-012	`MicrosToTimestamp` — conversion from microseconds

Requirement	Coverage
REQ-TIME-013	`MillisToTimestamp` — conversion from milliseconds
REQ-TIME-014	`TimestampToDouble` — conversion to double
REQ-TIME-015	`DoubleToTimestamp` — conversion from double
REQ-TIME-016	`JunoTime_TimeApiInit` — API initialization macro

19. IO Module

19.1. Purpose

The IO module (`async_io_api.h`, `i2c_io_api.h`, `spi_io_api.h`) provides abstract hardware IO interfaces for asynchronous byte-stream IO, I2C bus transfers, and SPI transactions. All three are pure vtable interfaces — implementations are platform-specific and injected by the integrator.

19.2. Data Structures

19.2.1. Async IO Root (`JUNO_ASYNC_IO_ROOT_T`)

```
struct JUNO_ASYNC_IO_ROOT_TAG JUNO_MODULE_ROOT(JUNO_ASYNC_IO_API_T,
JUNO_MODULE_EMPTY);
```

19.2.2. Async IO API (`JUNO_ASYNC_IO_API_T`)

Function	Description
`Read(ptIo, pcBuff, pzBuffSize)`	Read up to `*pzBuffSize` bytes; updates size with bytes read.
`TryRead(ptIo, pcBuff, pzBuffSize, iTimeoutUs)`	Read with timeout.
`ReadUntil(ptIo, pcBuff, pzBuffSize, pcStopChars, zSizeStopChars)`	Read until stop character.
`TryReadUntil(…)`	Read until stop character with timeout.
`Write(ptIo, pvBuff, pzBuffSize)`	Write up to `*pzBuffSize` bytes; updates size with bytes written.
`TryWrite(ptIo, pvBuff, pzBuffSize, iTimeoutUs)`	Write with timeout.
`Poll(ptIo, iTimeoutUs, pbHasData)`	Poll for input readiness.

Size parameters are in/out: on input they specify capacity; on return they reflect actual bytes

transferred.

19.2.3. I2C IO Root and Messages

`JUNO_I2C_IO_ROOT_T`

Module root for I2C.

`JUNO_I2C_IO_MSG_T`

Union of read/write message descriptors sharing a common header.

`JUNO_I2C_IO_MSG_HDR_T`

`{tType, iAddr}` — message type enum and 8-bit device address.

`JUNO_I2C_IO_MSG_R_T`

`{tHdr, ptReadBuff, zReadBuffSize}` — read descriptor.

`JUNO_I2C_IO_MSG_W_T`

`{tHdr, ptWriteBuff, zWriteBuffSize}` — write descriptor.

19.2.4. I2C IO API (`JUNO_I2C_IO_API_T`)

Single function: `Transfer(ptI2c, ptMsgs, zNumMsgs)` — executes an array of I2C messages.

19.2.5. I2C Helper Macros

`ReadMsg(iAddr, pcBuff, zBuffSize)`

Constructs a read message literal.

`WriteMsg(iAddr, pvBuff, zBuffSize)`

Constructs a write message literal.

`JUNO_I2C_IO_TRANSFER(…)`

Wraps messages into a compound literal array.

19.2.6. SPI IO Root and API

`JUNO_SPI_IO_ROOT_T`

Module root for SPI.

`JUNO_SPI_IO_API_T`

Single function: `Transaction(ptIo, pcReadBuff, zReadBuffSize, pvWriteBuff, zWriteBuffSize)` — full-duplex SPI transaction.

19.3. Interface Design

All three IO modules are pure abstract interfaces — no LibJuno-provided implementations. Integrators create platform-specific derivations that implement the vtable for their hardware.

19.4. Algorithm Descriptions

No algorithms — the IO modules define interfaces only. Implementation algorithms depend on the target platform (UART DMA, I2C peripheral registers, SPI controller).

19.5. Error Handling

- All functions return `JUNO_STATUS_T` for consistent error propagation.
- Timeout variants return `JUNO_STATUS_TIMEOUT_ERROR` when the deadline expires.
- Read/Write failures return `JUNO_STATUS_READ_ERROR` or `JUNO_STATUS_WRITE_ERROR`.
- The module root carries a failure handler for diagnostic reporting.

19.6. Design Rationale

IO interfaces are intentionally minimal — they define the contract, not the implementation. Hardware IO is inherently platform-specific; the vtable pattern allows injecting UART, USB, network, or mock implementations without changing application code. The I2C message-array pattern (`Transfer` with an array of read/write descriptors) mirrors common I2C hardware APIs (Linux `i2c_msg`, Zephyr `i2c_msg`) and enables multi-message transactions (write-then-read) in a single call.

19.7. Requirements Traceability

Requirement	Coverage
REQ-IO-001	`JUNO_ASYNC_IO_ROOT_T` — async IO root definition
REQ-IO-002	`Read` — byte-stream read
REQ-IO-003	`TryRead` — read with timeout
REQ-IO-004	`ReadUntil` — read until stop character
REQ-IO-005	`TryReadUntil` — read until stop char with timeout
REQ-IO-006	`Write` — byte-stream write
REQ-IO-007	`TryWrite` — write with timeout
REQ-IO-008	`Poll` — input readiness polling
REQ-IO-009	`JUNO_I2C_IO_ROOT_T` — I2C root definition
REQ-IO-010	`Transfer` — I2C multi-message transaction
REQ-IO-011	`JUNO_I2C_IO_MSG_HDR_T` — message header with type/address
REQ-IO-012	`JUNO_I2C_IO_MSG_T` — read/write message union
REQ-IO-013	`JUNO_SPI_IO_ROOT_T` — SPI root definition
REQ-IO-014	`Transaction` — full-duplex SPI transaction

20. Log Module

20.1. Purpose

The Log module (`log_api.h`) provides a minimal, pluggable logging facade for embedded targets. It defines four severity levels (Debug, Info, Warning, Error) as vtable functions. Implementations decide the sink (UART, RTT, buffer) and formatting strategy.

20.2. Data Structures

20.2.1. Log Root (`JUNO_LOG_ROOT_T`)

```
struct JUNO_LOG_ROOT_TAG JUNO_MODULE_ROOT(JUNO_LOG_API_T, JUNO_MODULE_EMPTY);
```

20.2.2. Log API (`JUNO_LOG_API_T`)

Function	Description
`LogDebug(ptJunoLog, pcMsg, …)`	Log a debug-level message (most verbose).
`LogInfo(ptJunoLog, pcMsg, …)`	Log an info-level message.
`LogWarning(ptJunoLog, pcMsg, …)`	Log a warning-level message.
`LogError(ptJunoLog, pcMsg, …)`	Log an error-level message.

All functions accept variadic arguments (printf-like format).

20.3. Interface Design

`JunoLog_LogInit(ptLog, ptApi, pfcnFailureHandler, pvUserData)`

Inline function that wires the log API and failure handler into the root. Returns `JUNO_STATUS_SUCCESS` on success; `JUNO_STATUS_NULLPTR_ERROR` if `ptLog` is NULL.

20.4. Algorithm Descriptions

No algorithms — the Log module defines the interface only. Formatting and output are implementation-specific.

20.5. Error Handling

- `JunoLog_LogInit` validates the log pointer via `JUNO_ASSERT_EXISTS`.
- All log functions return `JUNO_STATUS_T` for consistent error propagation.

- The module root carries a failure handler for self-diagnosis.
- Format string interpretation is implementation-defined; callers should avoid undefined specifiers in freestanding builds.

20.6. Design Rationale

The logging interface is minimal (four severity levels, printf-like format) to remain portable and lightweight. No filtering, buffering, or thread-safety guarantees are imposed at the interface level — these are implementation concerns that vary drastically between targets.

20.7. Requirements Traceability

Requirement	Coverage
REQ-LOG-001	`JUNO_LOG_ROOT_T` — log root definition
REQ-LOG-002	`LogDebug` — debug-level logging
REQ-LOG-003	`LogInfo` — info-level logging
REQ-LOG-004	`LogWarning` — warning-level logging
REQ-LOG-005	`LogError` — error-level logging
REQ-LOG-006	`JunoLog_LogInit` — initialization with API and failure handler
REQ-LOG-007	Printf-like variadic format for all levels

21. Mutex Module

21.1. Purpose

The Mutex module (`mtx_api.h`) provides a minimal mutual exclusion interface for cooperative concurrency. It defines TryLock, Lock, and Free operations as vtable functions. Implementations provide platform-specific locking semantics.

21.2. Data Structures

21.2.1. Mutex Root (`JUNO_MP_MTX_ROOT_T`)

```
struct JUNO_MP_MTX_ROOT_TAG JUNO_MODULE_ROOT(JUNO_MP_MTX_API_T, JUNO_MODULE_EMPTY);
```

21.2.2. Mutex API (`JUNO_MP_MTX_API_T`)

Function	Description
`TryLock(ptMtx)`	Non-blocking lock attempt. Returns `JUNO_RESULT_BOOL_T` (true if acquired).
`Lock(ptMtx)`	Blocking lock (may busy-wait). Returns `JUNO_STATUS_T`.

Function	Description
`Free(ptMtx)`	Release the lock. Returns `JUNO_STATUS_T`.

21.3. Interface Design

`JunoMp_MtxVerify(ptMtx)`

Inline verification that the mutex root, API pointer, and all three function pointers (TryLock, Lock, Free) are non-NULL.

No `Init` function is defined at the interface level — initialization is implementation-specific (e.g., platform mutex creation).

21.4. Algorithm Descriptions

No algorithms — the Mutex module defines the interface only. Lock/unlock semantics depend on the platform (spin lock, OS mutex, test-and-set).

21.5. Error Handling

- `JunoMp_MtxVerify` returns `JUNO_STATUS_NULLPTR_ERROR` if any required pointer is NULL.
- `TryLock` returns a Result type with boolean success indicator.
- `Lock` and `Free` return status codes.

21.6. Design Rationale

The mutex interface is deliberately minimal — three operations covering the essential lock lifecycle. No priority inheritance, recursive locking, or timeout parameters are specified at the interface level because these semantics vary fundamentally between platforms. The vtable pattern allows injecting platform mutexes (FreeRTOS, Zephyr, POSIX, bare-metal spin locks) or test doubles.

21.7. Requirements Traceability

Requirement	Coverage
REQ-MTX-001	`JUNO_MP_MTX_ROOT_T` — mutex root definition
REQ-MTX-002	`TryLock` — non-blocking lock attempt
REQ-MTX-003	`Lock` — blocking lock acquisition
REQ-MTX-004	`Free` — lock release
REQ-MTX-005	`JunoMp_MtxVerify` — mutex verification

22. State Machine Module

22.1. Purpose

The State Machine (SM) module (`sm_api.h`) provides a minimal state machine framework where each state is a module with actions, exit checks, and explicit transitions. The SM root holds the current state pointer; transitions occur via an optional next-state stored in each state root.

22.2. Data Structures

22.2.1. SM Root (`JUNO_SM_ROOT_T`)

```
struct JUNO_SM_ROOT_TAG JUNO_MODULE_ROOT(void,
    JUNO_SM_STATE_ROOT_T *ptCurrentState;  // Pointer to the active state
);
```

Note: the SM root uses `void` as the API type — it has no vtable of its own. All dispatch goes through the state's API.

22.2.2. State Root (`JUNO_SM_STATE_ROOT_T`)

```
struct JUNO_SM_STATE_ROOT_TAG JUNO_MODULE_ROOT(JUNO_SM_STATE_API_T,
    JUNO_SM_ROOT_T *ptSm;                          // Back-pointer to owning SM
    JUNO_SM_OPTION_STATE_T tOptionNextState;        // Optional next state
);
```

22.2.3. State API (`JUNO_SM_STATE_API_T`)

Function	Description
`StateAction(ptState)`	Execute the state's behavior.
`ShouldExit(ptState)`	Return whether the state should exit (Result<bool>).
`ResetState(ptState)`	Reset any state-local data.

22.2.4. Result/Option Types

`JUNO_SM_RESULT_STATE_T`

Result carrying a state pointer.

`JUNO_SM_OPTION_STATE_T`

Option carrying a state pointer.

`JUNO_SM_RESULT_OPTION_STATE_T`

Result carrying an option of a state pointer.

22.3. Interface Design

`JunoSm_Init(ptSmRoot, ptStartState, pfcnFailureHandler, pvFailureUserData)`

Initializes the SM with a start state. Validates via `JunoSm_Verify`.

`JunoSm_StateInit(ptSm, ptStateRoot, ptNextState, ptStateApi, pfcnFailureHandler, pvFailureUserData)`

Initializes a state, linking it to the SM and an optional next state. If `ptNextState` is NULL, the option is set to `NONE`.

`JunoSm_GetCurrentState(ptSmRoot)`

Returns a Result containing the current state pointer.

`JunoSm_TransitionState(ptSmRoot)`

If the current state has a next state (option is `SOME`), transitions the SM to that state. Returns a Result<Option<State>> — `SOME` with the new state, or `NONE` if no transition occurred.

`JunoSm_Verify(ptSmRoot)`

Validates SM root is non-NULL and has a current state.

`JunoSm_StateVerify(ptSmState)`

Validates state root, API, back-pointer, and next-state pointer (if present).

22.4. Algorithm Descriptions

22.4.1. State Machine Execution Loop (User-Driven)

The SM module does not enforce an execution loop — the user drives it:

1. Call `GetCurrentState` to get the active state.
2. Call `StateAction` on the state.
3. Call `ShouldExit` to check for transition.
4. If exiting: call `TransitionState` to move to the next state.
5. If next state is `SOME`: call `ResetState` on the new state.

This gives the user full control over when and how often to step the machine.

22.4.2. Transition

1. Read `ptCurrentState→tOptionNextState`.
2. If `bIsSome`: set `ptSmRoot→ptCurrentState = tSome`. Return `SOME(newState)`.
3. If not: return `NONE`. No change.

22.5. Error Handling

- `JunoSm_Init` returns `JUNO_STATUS_NULLPTR_ERROR` if the SM root or start state is NULL.
- `JunoSm_StateInit` returns `JUNO_STATUS_NULLPTR_ERROR` if required parameters are NULL.
- `JunoSm_GetCurrentState` propagates `Verify` errors.
- `JunoSm_TransitionState` propagates `Verify` errors. Returns `NONE` option (not error) when no next state is set.
- All functions use `JUNO_ASSERT_EXISTS` for precondition checking.

22.6. Design Rationale

The SM is deliberately minimal — it provides state storage, transitions, and verification but does not enforce an execution model. The user calls `StateAction`, `ShouldExit`, and `TransitionState` explicitly, allowing integration into any scheduling scheme (cyclic executive, event-driven, RTOS task). States are full module roots with their own failure handlers, enabling per-state error reporting.

22.7. Requirements Traceability

Requirement	Coverage
REQ-SM-001	`JUNO_SM_ROOT_T` — SM root with current state pointer
REQ-SM-002	`JUNO_SM_STATE_ROOT_T` — state root with SM back-pointer and next-state option
REQ-SM-003	`JUNO_SM_STATE_API_T` — state vtable (Action, ShouldExit, ResetState)
REQ-SM-004	`JunoSm_Init` — SM initialization
REQ-SM-005	`JunoSm_StateInit` — state initialization with linking
REQ-SM-006	`JunoSm_GetCurrentState` — current state access
REQ-SM-007	`JunoSm_TransitionState` — state transition
REQ-SM-008	`JunoSm_TransitionState` — NONE when no next state
REQ-SM-009	`JunoSm_Verify` — SM root validation
REQ-SM-010	`JunoSm_StateVerify` — state root validation
REQ-SM-011	`JunoSm_StateVerify` — next-state pointer validation

23. Software Bus (Broker) Module

23.1. Purpose

The Software Bus module (`broker_api.h`) provides a single-threaded publish/subscribe message broker. It solves intra-thread message fan-out: "A message arrived in my thread — who needs a

copy?" The broker maintains a registry of subscriber pipes (queues) keyed by Message ID (MID) and fans out published messages to all matching pipes.

23.2. Data Structures

23.2.1. Broker Root (JUNO_SB_BROKER_ROOT_T)

```
struct JUNO_SB_BROKER_ROOT_TAG JUNO_MODULE_ROOT(JUNO_SB_BROKER_API_T,
    JUNO_SB_PIPE_T **ptPipeRegistry;  // Array of pipe pointers
    size_t zRegistryLength;            // Current registered pipe count
    size_t zRegistryCapacity;          // Max pipes
);
```

23.2.2. Pipe (JUNO_SB_PIPE_T)

```
struct JUNO_SB_PIPE_TAG JUNO_MODULE_DERIVE(JUNO_DS_QUEUE_ROOT_T,
    JUNO_SB_MID_T iMsgId;  // Message ID / topic
);
```

A pipe is a queue derivation carrying a topic identifier. It inherits all queue operations (Enqueue, Dequeue, Peek) via the queue root.

23.2.3. Message ID (JUNO_SB_MID_T)

`uint32_t` topic identifier used to route messages to matching pipes.

23.2.4. Broker API (JUNO_SB_BROKER_API_T)

Function	Description
`Publish(ptBroker, tMid, tMsg)`	Enqueue `tMsg` into all pipes matching `tMid`.
`RegisterSubscriber(ptBroker, ptPipe)`	Add a pipe to the registry.

23.3. Interface Design

`JunoSb_BrokerInit(ptBroker, ptPipeRegistry, iRegistryCapacity, pfcnFailureHdlr, pvFailureUserData)`

Initializes the broker with a caller-owned pipe registry array. `zRegistryLength` starts at 0.

`JunoSb_PipeInit(ptPipe, iMid, ptArray, pfcnFailureHandler, pvUserData)`

Inline function that sets the pipe's MID and delegates to `JunoDs_QueueInit` for the underlying queue.

`JunoSb_BrokerVerify(ptBroker)`

Validates broker root, API (Publish, RegisterSubscriber), pipe registry pointer, and capacity.

`JunoSb_BrokerApiVerify(ptBrokerApi)`

Validates that Publish and RegisterSubscriber are non-NULL.

`JunoSb_PipeVerify(ptPipe)`

Validates the pipe and delegates to `JunoDs_QueueVerify` for the underlying queue.

23.4. Algorithm Descriptions

23.4.1. Publish

1. For each pipe in the registry (`0..zRegistryLength`):
   a. If `pipe→iMsgId == tMid`: enqueue `tMsg` into the pipe via the queue API.
2. Return status (propagate first error encountered).

Complexity: O(k) where k = registry length.

23.4.2. RegisterSubscriber

1. Check capacity: if `zRegistryLength >= zRegistryCapacity`, return `JUNO_STATUS_TABLE_FULL_ERROR`.
2. Store pipe pointer in `ptPipeRegistry[zRegistryLength]`.
3. Increment `zRegistryLength`.

Complexity: O(1).

23.5. Error Handling

- `Publish` propagates queue enqueue errors (e.g., pipe full → `JUNO_STATUS_INVALID_SIZE_ERROR`).
- `RegisterSubscriber` returns `JUNO_STATUS_TABLE_FULL_ERROR` when the registry is full.
- All operations call `Verify` at entry.
- `JunoSb_PipeInit` propagates `JunoDs_QueueInit` errors.

23.6. Design Rationale

Single-threaded by design. IPC mechanisms are platform-specific and non-abstractable without losing the properties that make them useful (blocking semantics, copy vs. zero-copy, priority inheritance, WCET guarantees). The broker solves intra-thread fan-out portably; inter-thread transport belongs to the integrator.

Pipes (queues) over callbacks. Three reasons:

1. **Temporal decoupling** — `Publish` does O(1) enqueue per pipe; subscribers process on their own schedule.

2. **Bounded backpressure** — each pipe has fixed capacity; overflow returns an error status.
3. **Subscriber control** — subscribers inspect, batch, and prioritize message consumption.

Multi-threaded integration pattern. One broker per thread, bridged by a user-written bridge app that forwards messages between brokers via platform IPC. The broker is a routing table, not a transport.

23.7. Requirements Traceability

Requirement	Coverage
REQ-SB-001	`JUNO_SB_BROKER_ROOT_T` — broker root with pipe registry
REQ-SB-002	`JUNO_SB_BROKER_API_T` — Publish and RegisterSubscriber vtable
REQ-SB-003	Publish fan-out to matching MIDs
REQ-SB-004	`JUNO_SB_PIPE_T` — pipe derivation of queue with MID
REQ-SB-005	`JunoSb_PipeInit` — pipe initialization
REQ-SB-006	Subscriber dequeue via queue API
REQ-SB-007	Registry capacity enforcement
REQ-SB-008	Bounded pipe capacity (queue-based backpressure)
REQ-SB-009	Single-threaded scope
REQ-SB-010	`JunoSb_BrokerVerify` — broker validation
REQ-SB-011	`JunoSb_BrokerVerify` — registry pointer and capacity check
REQ-SB-012	`JunoSb_PipeVerify` — pipe validation
REQ-SB-013	Thread-safety documentation and scope declaration

24. Application Module

24.1. Purpose

The Application module (`app_api.h`) defines a minimal application lifecycle interface: `OnStart`, `OnProcess`, and `OnExit`. It provides the unit of schedulable work for the Scheduler and a common interface for application composition.

24.2. Data Structures

24.2.1. Application Root (`JUNO_APP_ROOT_T`)

```
struct JUNO_APP_ROOT_TAG JUNO_MODULE_ROOT(JUNO_APP_API_T, JUNO_MODULE_EMPTY);
```

The application root carries only the standard module fields (ptApi, failure handler, user data).

Application-specific state is added via derivation.

24.2.2. Application API (`JUNO_APP_API_T`)

Function	Description
`OnStart(ptJunoApp)`	Called once to initialize resources before processing.
`OnProcess(ptJunoApp)`	Main processing routine; may be invoked repeatedly by a scheduler.
`OnExit(ptJunoApp)`	Called once prior to shutdown to release resources.

24.3. Interface Design

No `Init` function is defined at the interface level. Applications are initialized by:

1. Allocating an `JUNO_APP_ROOT_T` (or derivation).
2. Setting `ptApi` to a `static const JUNO_APP_API_T` vtable.
3. Setting failure handler fields as needed.

The Scheduler calls `OnProcess` on each app in each minor frame. `OnStart` and `OnExit` are called once at system startup and shutdown respectively.

24.4. Algorithm Descriptions

No algorithms — the Application module defines a lifecycle contract only.

24.5. Error Handling

- All lifecycle functions return `JUNO_STATUS_T` for status propagation.
- The module root carries a failure handler for diagnostic reporting.
- The Scheduler propagates errors returned by `OnProcess`.

24.6. Design Rationale

The Application module is the thinnest possible lifecycle abstraction — three hooks covering the universal start/run/stop pattern. It does not prescribe initialization strategy, thread ownership, or resource management — these are implementation concerns handled by derivations. The module root pattern enables the Scheduler to call `OnProcess` through a uniform `JUNO_APP_ROOT_T *` without knowing the concrete application type.

24.7. Requirements Traceability

Requirement	Coverage
REQ-APP-001	`JUNO_APP_ROOT_T` — application root definition

Requirement	Coverage
REQ-APP-002	`OnStart` — initialization lifecycle hook
REQ-APP-003	`OnProcess` — main processing lifecycle hook
REQ-APP-004	`OnExit` — shutdown lifecycle hook

25. Scheduler Module

25.1. Purpose

The Scheduler module (`juno_sch_api.h`) provides a table-driven cyclic executive that executes application `OnProcess` hooks in minor frames of fixed duration. It is the top-level framework component that composes Applications, Time, and the module system into a deterministic execution schedule.

25.2. Data Structures

25.2.1. Scheduler Root (`JUNO_SCH_ROOT_T`)

```
struct JUNO_SCH_ROOT_TAG JUNO_MODULE_ROOT(JUNO_SCH_API_T,
    JUNO_TIME_ROOT_T *ptTime;            // Injected time source
    JUNO_APP_ROOT_T **ptArrSchTable;     // Flattened 2D schedule table
    size_t zAppsPerMinorFrame;            // Slots per minor frame
    size_t zNumMinorFrames;               // Minor frames per major frame
    JUNO_TIMESTAMP_T tMinorFramePeriod;   // Duration of each minor frame
);
```

25.2.2. Scheduler API (`JUNO_SCH_API_T`)

Function	Description
`Execute(ptJunoSch)`	Run the scheduler for one major frame.
`GetMinorFramePeriod(ptJunoSch)`	Return the configured minor frame period.
`GetMajorFramePeriod(ptJunoSch)`	Compute and return the major frame period.

25.2.3. Schedule Table Macro

`JUNO_SCH_TABLE_NEW(ptArrName, zAppsPerMinorFrame, zNumMinorFrames, …)`

Declares a static array of `JUNO_APP_ROOT_T *` with the given dimensions. NULL entries indicate empty slots in variable-length frames.

25.3. Interface Design

The scheduler is initialized by populating a `JUNO_SCH_ROOT_T` with:

1. A time module instance (for `Now`, `SleepTo`, time math).
2. A flattened schedule table array.
3. Frame dimensions and period.

`Execute` runs one major frame (all minor frames in sequence).

25.4. Algorithm Descriptions

25.4.1. Execute (One Major Frame)

```
tFrameStart = Now()
for each minor frame i in [0, zNumMinorFrames):
    tDeadline = tFrameStart + tMinorFramePeriod * (i + 1)
    for each slot j in [0, zAppsPerMinorFrame):
        ptApp = ptArrSchTable[i * zAppsPerMinorFrame + j]
        if ptApp != NULL:
            ptApp->ptApi->OnProcess(ptApp)
    SleepTo(tDeadline)
```

1. Get the major frame start time via `Now`.
2. For each minor frame, compute the deadline by adding `(i + 1) * tMinorFramePeriod`.
3. Execute all non-NULL apps in the minor frame sequentially.
4. Sleep until the minor frame deadline via `SleepTo`.

Complexity: O(F × A) per major frame, where F = minor frames and A = apps per frame.

25.4.2. GetMajorFramePeriod

Computes `tMinorFramePeriod x zNumMinorFrames` using `AddTime`.

25.4.3. Flattened 2D Indexing

The schedule table is a 1D array with index arithmetic:

\text{index}(i, j) = i \times \texttt{zAppsPerMinorFrame} + j

where i is the minor frame index and j is the app slot within the frame. NULL entries indicate unused slots, allowing variable-length minor frames within a fixed-width table.

25.5. Error Handling

- `Execute` propagates errors from `Now`, `AddTime`, `SleepTo`, and each `OnProcess` call.
- NULL app pointers in the table are silently skipped (not errors).
- `GetMinorFramePeriod` and `GetMajorFramePeriod` return Result types with status codes.

25.6. Design Rationale

Static cyclic executive. Deterministic timing by construction — execution order and frame boundaries are fixed at compile time. No OS dependency (works on bare metal with a single timer). Certifiable by inspection (DO-178C, MISRA, IEC 61508). The scheduler complements RTOS preemption rather than replacing it — it fills the intra-thread scheduling gap.

Flattened 2D array. Zero dynamic allocation (single caller-owned buffer), cache-friendly sequential access, O(1) index arithmetic, NULL slots for variable-length frames without complex data structures.

Injected Time module. Clock hardware (`Now`) and blocking primitives (`Sleep`, `SleepTo`) are platform-specific. Time math (add, subtract, conversions) is universal and provided by LibJuno. The split follows the DI principle: inject what varies, provide what's universal.

25.7. Requirements Traceability

Requirement	Coverage
REQ-SCH-001	`JUNO_SCH_ROOT_T` — scheduler root with time and table dependencies
REQ-SCH-002	`JUNO_SCH_API_T` — scheduler vtable (Execute, periods)
REQ-SCH-003	`JUNO_SCH_TABLE_NEW` — static table declaration macro
REQ-SCH-004	`Execute` — one major frame execution loop
REQ-SCH-005	`GetMinorFramePeriod` — period query
REQ-SCH-006	`GetMajorFramePeriod` — computed major frame period
REQ-SCH-007	Time module dependency injection

26. Data Design

26.1. Common Types

LibJuno defines a small set of common value types used across all modules:

`JUNO_STATUS_T` (`int32_t`) — the unified error code type. Zero indicates success; non-zero values indicate specific failure conditions.

Result types (`JUNO_MODULE_RESULT(NAME_T, OK_T)`) — structs combining `{JUNO_STATUS_T tStatus; OK_T tOk;}`. Pre-defined specializations: `JUNO_RESULT_BOOL_T`, `JUNO_RESULT_UINT32_T`, `JUNO_RESULT_F64_T`,

`JUNO_RESULT_SIZE_T`, `JUNO_RESULT_VOID_PTR_T`.

Option types (`JUNO_MODULE_OPTION(NAME_T, SOME_T)`) — structs combining `{bool bIsSome; SOME_T tSome;}`. Used for operations where absence is a legitimate state (peek on empty collection, optional next-state in SM).

`JUNO_POINTER_T` — the fat pointer descriptor `{ptApi, pvAddr, zSize, zAlignment}`. Carries self-describing type information and operations for type-safe memory access.

26.2. Memory Ownership Model

All memory in LibJuno is **caller-owned and injected**:

1. The caller allocates storage (stack, static, or platform-specific memory pool).
2. The caller passes storage to an `Init` function along with all dependencies.
3. The module operates exclusively on the injected storage.
4. The caller is responsible for the lifetime of all injected memory.

No module in LibJuno calls `malloc`, `calloc`, `realloc`, or `free`. Memory pools (block allocator) operate on caller-provided backing buffers.

26.3. Design Rationale: Result/Option Types and Error Handling

Why Rust-style Result/Option in C11. `errno` is a hosted-only thread-local global unavailable in freestanding builds and violates the "no global mutable state" constraint. Out-parameters separate the value from its validity — nothing prevents ignoring the status and using a garbage out-parameter. Result structs bind the value and status in a single aggregate with `JUNO_ASSERT_OK` enforcing check-before-use.

Why failure handlers are diagnostic-only. Three reasons:

1. **Predictable WCET** — if handlers could `longjmp` or abort, the control flow of every library function becomes unanalyzable.
2. **No stack unwinding** — `longjmp`/`setjmp` is hosted-only; exceptions require RTTI. The only portable freestanding flow control is "check status, return status."
3. **Separation of policy from mechanism** — the library decides what constitutes an error; the application decides what to do about it (log, blink LED, set fault flag).

Why the ASSERT macro family. Each macro maps to a specific domain of validation:

- `JUNO_ASSERT_EXISTS(ptr)` → always returns `JUNO_STATUS_NULLPTR_ERROR` — correct specific status.
- `JUNO_ASSERT_SUCCESS(s, ⋯)` → variadic action statements for ergonomic propagation.
- `JUNO_ASSERT_OK(result, ⋯)` → unwraps Result types.
- `JUNO_ASSERT_SOME(opt, ⋯)` → unwraps Option types.

Each macro name is self-documenting: it reads as a named proof obligation.

27. Interface Design

27.1. Module Initialization Contracts

Every module follows the same initialization pattern:

1. Caller allocates all storage.
2. Caller calls `Module_Init(…)` passing the instance, all dependencies, and an optional failure handler.
3. `Init` wires the vtable, stores dependencies, and calls `Verify`.
4. `Verify` validates all pointers and dependencies are non-NULL.
5. All subsequent public functions call `Verify` at entry.

```
JUNO_STATUS_T JunoDs_Heap_Init(
    JUNO_DS_HEAP_ROOT_T *ptHeap,
    const JUNO_DS_HEAP_POINTER_API_T *ptHeapPointerApi,
    JUNO_DS_ARRAY_ROOT_T *ptHeapArray,
    JUNO_FAILURE_HANDLER_T pfcnFailureHdlr,
    JUNO_USER_DATA_T *pvUserData
);
```

27.2. Vtable Dispatch Pattern

API dispatch follows this pattern:

```
// Caller invokes via vtable:
ptModule->ptApi->DoSomething(&ptModule->tRoot, args...);

// Or through the root directly:
JUNO_MODULE_GET_API(ptModule, ROOT_T)->DoSomething((ROOT_T *)ptModule, args...);
```

The vtable is `const` and typically `static` — enabling ROM placement. Dispatch requires one pointer dereference through `ptApi`, then a function pointer call.

27.3. Trait System (JUNO_TRAIT_ROOT)

Traits are lightweight interfaces without failure handler fields:

```
struct JUNO_POINTER_TAG JUNO_TRAIT_ROOT(JUNO_POINTER_API_T,
    void *pvAddr;
    size_t zSize;
```

```
    size_t zAlignment;
);
```

27.4. Design Rationale: Pointer Trait System

Why a fat pointer with a vtable. `JUNO_POINTER_T` binds `{pvAddr, zSize, zAlignment, ptApi}` into a self-describing, self-validating value type. In freestanding environments, `memcpy` is not guaranteed to exist; by putting `Copy` behind a vtable in the pointer, the element knows how to copy itself — data structures never need `memcpy`.

Why Copy/Reset on the pointer, not per-data-structure. One Copy/Reset implementation per element type, reused by every data structure (heap, queue, stack, map). Data structures remain type-agnostic; adding a new element type requires only a new `JUNO_POINTER_API_T` implementation.

Why `JUNO_TRAIT_ROOT` (no failure handler). The pointer is a pass-by-value ephemeral descriptor, not a stateful module instance. Adding failure handler fields (+16 bytes on 64-bit) would bloat every data structure call's stack frame. Failure reporting belongs to the owning module.

27.5. Software Bus Integration Pattern

The broker provides single-threaded publish/subscribe message fan-out:

```
// Publisher:
ptBrokerApi->Publish(ptBroker, ENGINE_CMD_MID, tMsgPointer);

// Subscriber (in its own OnProcess):
ptQueueApi->Dequeue(&ptCmdPipe->tRoot, tReturnPointer);
```

27.6. Design Rationale: Single-Threaded Broker

Why single-threaded scope. IPC mechanisms are platform-specific and non-abstractable without losing the properties that make them useful (blocking semantics, copy vs. zero-copy, priority inheritance, WCET guarantees). The broker solves intra-thread fan-out portably; inter-thread transport belongs to the integrator.

Why pipes (queues) over callbacks. Three reasons:

1. **Temporal decoupling** — Publish does O(1) enqueue; subscriber processes on its own schedule.
2. **Bounded backpressure** — each pipe has fixed capacity; overflow returns an error status.
3. **Subscriber control** — subscribers inspect, batch, and prioritize message consumption.

Multi-threaded integration pattern. One broker per thread, bridged by a user-written bridge app that forwards messages between brokers via platform IPC.

27.7. Scheduler Integration Pattern

The scheduler is a table-driven cyclic executive:

```
JUNO_SCH_TABLE_NEW(ptTable, 2, 3,
    &tAppA.tRoot, &tAppB.tRoot,   // Minor frame 0
    &tAppA.tRoot, NULL,            // Minor frame 1
    &tAppC.tRoot, &tAppB.tRoot,   // Minor frame 2
);
```

27.8. Design Rationale: Cyclic Executive

Why static cyclic executive. Deterministic timing by construction — execution order and boundaries are fixed at compile time. No OS dependency (works on bare metal with a single timer). Certifiable by inspection (DO-178C, MISRA, IEC 61508). Complements RTOS preemption rather than replacing it.

Why flattened 2D array. Zero allocation (single caller-owned buffer), cache-friendly sequential access, O(1) index arithmetic, NULL slots for variable-length frames.

Why injected Time module. Clock hardware (`Now`) and blocking primitives (`Sleep`, `SleepTo`) are platform-specific. Time math (add, subtract, conversions) is universal and provided by LibJuno. The split follows DI: inject what varies, provide what's universal.